Life at Low Reynolds Number

A Gentle Introduction to Stokes Flow, Spheres, and Their Interactions

Summer reading for a fourth-year project
in interfacial fluid dynamics

Before you begin

A word on why this booklet is written the way it is. Artificial intelligence is transforming how scientific code gets written — and, increasingly, how analytic mathematics gets done. A modern AI assistant will draft a solver, derive an identity, or refactor a messy script in seconds. This is genuinely powerful, and you should use it. But it must be treated with real wariness: these tools are fluent and confident even when they are wrong, and a plausible-looking wrong answer is far more dangerous than an obvious one.

The approach this booklet takes — and the habit it is really trying to teach — is to make **mathematical modelling and AI code generation work together**: use the mathematics you derive by hand to *validate* and *understand* the code an AI produces. The analysis is the anchor; the AI is the accelerator. Get this partnership right and you gain enormous speed without losing correctness. We hope this way of working will serve you well not just in this summer project, but in whatever career you choose.

Welcome. This short book is your warm-up for a fourth-year summer project in the fluid dynamics of small things — droplets, bubbles, thin films, and the particles that live in them. You have already met fluid dynamics in **MA3D1**: the Euler and Navier–Stokes equations, the stream function, vorticity, the stress tensor, potential flow, aerofoils, and boundary layers. That module lives at the *fast* end of fluid dynamics — high Reynolds number, where inertia rules and viscosity is confined to thin layers.

This book takes you to the opposite end: the *slow*, syrupy world of **Stokes flow**, where viscosity dominates and inertia is negligible. It is the world a bacterium swims in, the world a fog droplet settles in, and the world in which the interfacial flows you will study for your summer project actually take place.

Why spheres, and why no-slip? Our real interest is the *interaction of small liquid droplets* — for instance, water droplets in a cloud. Two facts about small droplets make the idealisation of this booklet the right one. First, when a droplet is small enough, **surface tension** dominates its shape: the energy cost of deforming the interface far outweighs the disturbance from the surrounding flow, so the droplet stays essentially *spherical*. Second, the liquid inside (water, say) has a viscosity roughly fifty times that of the surrounding air; the drop is so much stiffer to shear than the air around it that, from the air's point of view, its surface barely moves. To leading order we may therefore treat each droplet as a *rigid sphere with a no-slip surface*. That is exactly the object we spend Chapter 1 on, and the interaction of two of them is Chapters 2–3.

How to read it. The core results are worked in full — you should be able to follow every line with an MA3D1 background and a pen. But mathematics is not a spectator sport, so at each branching point you will find:

- **Exercises** (green boxes) — short, scaffolded, with full solutions in Appendix A. Do them.
- **Python & AI** (orange boxes) — short tasks to *build a piece of code* (with an AI assistant, if you like) and then check it against the result you just derived. They appear throughout the chapters, right next to the mathematics they test, rather than all at the end.

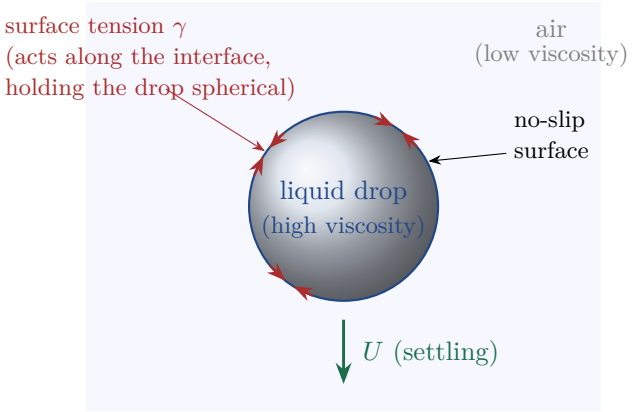


Figure 1: Why a rigid, no-slip sphere is the right model for a small droplet. The paired red arrows represent **surface tension** γ: it acts *along* the interface (each pair pulls towards the other), penalising any deformation and so holding the drop spherical. The liquid's high viscosity (relative to air) means its surface scarcely deforms under the external flow, so the interface acts as **no-slip**. The drop settles at speed U.

The plan. Chapter 0 bridges from what you know to what is coming. Chapter 1 derives the flow around a single sphere from scratch. Chapter 2 tackles the forces between *two* spheres. Chapter 3 builds a small but genuine computational tool — a two-sphere oracle in the spirit of Rosa *et al.* (2011) — that you can validate your own summer-project code against. The full source of the companion tools is collected in Appendix C; the Python & AI boxes along the way build it up piece by piece.

A note on the point of all this. During your summer project you will write code — very possibly with the help of an AI assistant. As above, these tools are a superb accelerator and a confident liar in equal measure. The single most valuable habit you can build this summer is: **never trust a number you cannot check against something you understand.** Every tool in this book exists to be that "something you understand". Guard it well.

Contents

Before you begin ii

0 From the fast limit to the slow limit 1
- 0.1 Just how small is small? . . . 1
- 0.2 The Stokes equations . . . 1
- 0.3 A tale of two limits . . . 2
- 0.4 A one-page primer on index notation . . . 3
- 0.5 The reciprocal theorem: the most useful idea in the book . . . 4

1 Flow around a single sphere 6
- 1.1 Set-up and symmetry . . . 6
- 1.2 From Stokes to the stream function . . . 6
- 1.3 Separable solution . . . 8
- 1.4 The velocity and pressure fields . . . 8
- 1.5 The drag: three ways to $6\pi\mu aU$ . . . 9
- 1.6 The long reach of Stokes flow . . . 10
- 1.7 The sphere as a sum of singularities . . . 11
- 1.8 Exercises . . . 12

2 Two spheres, and the forces between them 14
- 2.1 Resistance and mobility: forces as a linear map . . . 14
- 2.2 Far field I: the method of reflections . . . 15
- 2.3 Far field II: what the reflection series does and does not capture . . . 17
- 2.4 Near field: lubrication and the squeeze-film singularity . . . 17

3 A practical oracle: the Rosa method 20
- 3.1 The two ingredients . . . 20
- 3.2 What it produces . . . 21
- 3.3 A worked example: two droplets sedimenting . . . 22
- 3.4 Validation: the whole point . . . 23

A Solutions and hints 25

B Further reading 27

C The Python toolkit, in full 29
- C.1 `sphere_single.py` — the single-sphere oracle (Chapter 1) . . . 29
- C.2 `stokeslets.py` — singularity superposition and pair mobility (Chapters 1–2) . . . 33
- C.3 `two_sphere_rosa.py` — the two-sphere resistance oracle (Chapter 3) . . . 36
- C.4 `run_all_tests.py` — the unified test harness . . . 39

Chapter 0

From the fast limit to the slow limit

Everything in MA3D1 was, in a sense, the story of *one* number: the Reynolds number

$$\mathrm{Re} \;=\; \frac{\rho U L}{\mu} \;=\; \frac{\text{inertial forces}}{\text{viscous forces}}, \tag{0.1}$$

where ρ is the fluid density, μ its dynamic viscosity, and U, L a characteristic speed and length. Aerofoils, boundary layers, and vortex shedding all live at $\mathrm{Re} \gg 1$, where inertia dominates and viscosity is a thin-layer correction. This book lives at the other extreme, $\mathrm{Re} \ll 1$, where viscosity dominates everything and inertia is the correction you throw away. The physics could hardly be more different — and yet, reassuringly, almost every surprise you are about to meet has a mirror-image counterpart you already know.

0.1 Just how small is small?

It is worth feeling the numbers. Take water ($\mu \approx 10^{-3}\,\mathrm{Pa\,s}$, $\rho \approx 10^{3}\,\mathrm{kg\,m^{-3}}$).

Object	Size L	Speed U	Re
Airliner wing	$3\,\mathrm{m}$	$250\,\mathrm{m\,s^{-1}}$	$\sim 10^{8}$
Swimming human	$1\,\mathrm{m}$	$1\,\mathrm{m\,s^{-1}}$	$\sim 10^{6}$
Falling raindrop	$1\,\mathrm{mm}$	$1\,\mathrm{m\,s^{-1}}$	$\sim 10^{3}$
Settling fog droplet	$10\,\mu\mathrm{m}$	$3\,\mathrm{mm\,s^{-1}}$	$\sim 3\times10^{-2}$
Swimming bacterium	$2\,\mu\mathrm{m}$	$20\,\mu\mathrm{m\,s^{-1}}$	$\sim 4\times10^{-5}$

The last two rows are the world of this book. Notice that "small Re" comes overwhelmingly from *small size*, not from high viscosity: the same water that supports an airliner's boundary layer is, to a fog droplet, an almost unimaginably thick treacle. Reynolds himself made the point that what matters is the *combination* $\rho UL/\mu$, not any one factor.

0.2 The Stokes equations

Start from the incompressible Navier–Stokes equations you know,

$$\rho\left(\frac{\partial \boldsymbol{u}}{\partial t} + \boldsymbol{u}\cdot\boldsymbol{\nabla}\boldsymbol{u}\right) = -\boldsymbol{\nabla}p + \mu\nabla^{2}\boldsymbol{u}, \qquad \boldsymbol{\nabla}\cdot\boldsymbol{u} = 0. \tag{0.2}$$

Non-dimensionalise with the scales $\boldsymbol{x} = L\boldsymbol{x}^{*}$, $\boldsymbol{u} = U\boldsymbol{u}^{*}$, $t = (L/U)t^{*}$, and — this choice matters — the *viscous* pressure scale $p = (\mu U/L)p^{*}$. Dropping stars, the momentum equation becomes

$$\mathrm{Re}\left(\frac{\partial \boldsymbol{u}}{\partial t} + \boldsymbol{u}\cdot\boldsymbol{\nabla}\boldsymbol{u}\right) = -\boldsymbol{\nabla}p + \nabla^{2}\boldsymbol{u}. \tag{0.3}$$

Now let $\mathrm{Re} \to 0$. The entire left-hand side — *all* of the inertia, both the unsteady and the advective part — disappears, leaving the **Stokes equations** (or "creeping-flow" equations):

$$\boldsymbol{\nabla} p = \mu \nabla^2 \boldsymbol{u}, \tag{0.4}$$

$$\boldsymbol{\nabla} \cdot \boldsymbol{u} = 0. \tag{0.5}$$

(We have restored dimensions; you may keep or drop them as convenient.)

These two innocuous-looking equations have three structural properties that we will exploit relentlessly. They are worth stating up front, because nearly every result in the book is a consequence of one of them.

The three gifts of the Stokes equations

Linearity.
Equations (0.4)–(0.5) are *linear* in $\boldsymbol{u}$ and p. Solutions superpose. Double the boundary velocity and you double the whole flow, the pressure, and the force. This is what makes "forces $\leftrightarrow$ velocities" a *matrix* (Chapter 2) and what lets us build flows by adding elementary singularities (the Stokeslet, Chapter 1).

Instantaneity.
Time does not appear in (0.4)–(0.5). The flow has no memory and no inertia: it is slaved, *instantaneously*, to the current boundary conditions. Stop pushing and the fluid stops dead. The past is irrelevant; only the present configuration matters.

Reversibility.
Reverse the forcing ($\boldsymbol{u} \to -\boldsymbol{u}$, $p \to -p$) and you get an exact solution again. Run the boundary motion backwards and the fluid retraces its steps *exactly*. This "kinematic reversibility" is the source of the most delightful low-Re paradoxes.

0.3 A tale of two limits

Here is the map between the world you know and the world you are entering. Keep it to hand; the right-hand column is the whole book in miniature.

High Re **(MA3D1)**	**Low** Re **(this book)**
Drop the *viscous* term $\Rightarrow$ Euler equation.	Drop the *inertial* term $\Rightarrow$ Stokes equations.
d'Alembert's paradox: a body in steady ideal flow feels **zero** drag.	A sphere feels the finite **Stokes drag** $F = 6\pi\mu a U$ (Chapter 1).
Force on a 2D cylinder from the Blasius theorem; lift on an aerofoil.	**Stokes' paradox**: *no* bounded Stokes flow past an infinite 2D cylinder exists at all.
Disturbance of a moving body decays fast (dipole, $\sim 1/r^3$).	Disturbance decays **slowly** ($\sim 1/r$): Stokes flow is *long-ranged*. This is why two spheres interact so strongly (Chapter 2).
Superpose sources, sinks, and dipoles to build potential flows.	Superpose **Stokeslets**, dipoles, and stresslets to build Stokes flows (Chapters 1–3).
Kelvin circulation, vortex stretching, energy in the flow.	Kinematic **reversibility**; Purcell's "scallop theorem" forbids reciprocal swimmers.
Time-dependent wakes, shedding, turbulence.	**Instantaneous**, memoryless, and (usually) unique and stable.

Why "creeping" flow can still be subtle

Dropping inertia makes the equations linear, but it does *not* make the problems easy. The long range of the disturbance ($1/r$) means boundaries talk to each other across large distances; two settling spheres, or a sphere near a wall, are genuinely hard problems whose exact solutions occupied some of the finest applied mathematicians of the twentieth century. That tension — simple equations, rich behaviour — is exactly what makes this a good project area.

0.4 A one-page primer on index notation

Chapters 1–3 lean on *Cartesian index (suffix) notation*, because the elementary solutions of Stokes flow are naturally *tensors*. If you are fluent, skip this; if not, ten minutes here will pay for itself many times.

Write the components of a vector $\boldsymbol{u}$ as u_i, with $i \in \{1,2,3\}$ labelling the x, y, z directions. Two conventions do all the work.

Summation convention. A repeated index in a term is *summed* over $1, 2, 3$:

$$a_i b_i \;:=\; \sum_{i=1}^{3} a_i b_i = \boldsymbol{a}\cdot\boldsymbol{b}, \qquad \frac{\partial u_i}{\partial x_i} \;:=\; \sum_{i=1}^{3} \frac{\partial u_i}{\partial x_i} = \boldsymbol{\nabla}\cdot\boldsymbol{u}. \tag{0.6}$$

A repeated ("dummy") index is summed; a single ("free") index labels a component and must match on both sides of an equation.

Two special tensors. The Kronecker delta δ_{ij} (the identity, $= 1$ if $i = j$, else 0) and the alternating symbol ε_{ijk} (for cross products). You will mostly need δ_{ij}:

$$\delta_{ij} a_j = a_i, \qquad \delta_{ii} = 3, \qquad \frac{\partial x_i}{\partial x_j} = \delta_{ij}. \tag{0.7}$$

Gradients of r. With $r = \sqrt{x_k x_k} = |\boldsymbol{x}|$, two identities recur constantly (check them as Exercise 0.1):

$$\frac{\partial r}{\partial x_i} = \frac{x_i}{r}, \qquad \frac{\partial}{\partial x_j}\left(\frac{x_i}{r}\right) = \frac{\delta_{ij}}{r} - \frac{x_i x_j}{r^3}. \tag{0.8}$$

The stress tensor, in suffixes. The Newtonian stress you met in MA3D1 (which we write T, with components T_{ij}, throughout) is

$$T_{ij} = -p\,\delta_{ij} + \mu\left(\frac{\partial u_i}{\partial x_j} + \frac{\partial u_j}{\partial x_i}\right), \tag{0.9}$$

and the force the fluid exerts on a surface with outward unit normal $\boldsymbol{n}$ is the traction $\tau_i = T_{ij} n_j$, so the total force on a body is

$$F_i = \oint_S T_{ij} n_j \,\mathrm{d}S. \tag{0.10}$$

Equation (0.10) is how every drag in this book is ultimately computed — analytically *and*, in the Python tools, numerically.

Exercise 0.1

Warm up your index muscles.

(a) Show $\dfrac{\partial r}{\partial x_i} = \dfrac{x_i}{r}$ and hence $\dfrac{\partial}{\partial x_j}\left(\dfrac{x_i}{r}\right) = \dfrac{\delta_{ij}}{r} - \dfrac{x_i x_j}{r^3}$.

(b) Using (0.9), show that for an incompressible flow ($\partial u_k/\partial x_k = 0$) the divergence of the stress is $\partial T_{ij}/\partial x_j = -\partial p/\partial x_i + \mu\,\nabla^2 u_i$, so that the Stokes momentum equation (0.4) is simply $\partial T_{ij}/\partial x_j = 0$: *Stokes flow is mechanical equilibrium of every fluid element.*

(c) Deduce that $\varepsilon_{ijk} x_j\,(\partial T_{kl}/\partial x_l) = 0$ as well, and interpret it as balance of torques.

0.5 The reciprocal theorem: the most useful idea in the book

One consequence of linearity deserves its own section, because it is the single highest-leverage tool in low-Reynolds-number hydrodynamics. It lets you compute forces *without ever solving for the flow*.

Suppose the *same* geometry (the same rigid boundaries) supports two different Stokes flows: an "unhatted" flow $(\boldsymbol{u}, \mathsf{T})$ and a "hatted" flow $(\hat{\boldsymbol{u}}, \hat{\mathsf{T}})$, driven by different boundary motions. The **Lorentz reciprocal theorem** states that the two flows do equal work on each other across any bounding surface S:

$$\oint_S u_i\,\hat{T}_{ij}\,n_j\,\mathrm{d}S \;=\; \oint_S \hat{u}_i\,T_{ij}\,n_j\,\mathrm{d}S. \tag{0.11}$$

One-line proof (worth internalising)

Consider $\partial(u_i\hat{T}_{ij})/\partial x_j$. Expand: $u_i(\partial\hat{T}_{ij}/\partial x_j) + (\partial u_i/\partial x_j)\hat{T}_{ij}$. The first term vanishes (both flows satisfy $\partial T_{ij}/\partial x_j = 0$, Exercise 0.1b). The second, once you substitute the constitutive law (0.9) and use incompressibility, is *symmetric* under swapping hatted and unhatted fields: $(\partial u_i/\partial x_j)\hat{T}_{ij} = 2\mu\,e_{ij}\hat{e}_{ij} = (\partial\hat{u}_i/\partial x_j)T_{ij}$, where $e_{ij} = \frac{1}{2}(\partial_j u_i + \partial_i u_j)$ is the rate of strain. Hence $\partial(u_i\hat{T}_{ij})/\partial x_j = \partial(\hat{u}_i T_{ij})/\partial x_j$; integrate over the fluid volume and apply the divergence theorem to get (0.11).

Why care? Because (0.11) converts a hard question about one flow into an easy question about a flow you already know. We will use it three times:

- to prove that the grand resistance matrix of Chapter 2 is *symmetric* (so the force on sphere 1 due to the motion of sphere 2 is tied to the reverse);
- to derive **Faxén's laws**, which give the force and torque on a sphere sitting in *any* ambient flow, without re-solving;
- to extract drags and swimming speeds by pairing an unknown flow with the known single-sphere solution of Chapter 1.

Keep (0.11) in your pocket. When a low-Re calculation looks impossible, the reciprocal theorem is very often the way in.

Predict, then verify

No code yet — this box sets up the working habit for the ones that follow. Before Chapter 1, write down your own one-sentence answer to: *why does a sphere feel zero drag in ideal flow (d'Alembert) but finite drag $6\pi\mu aU$ in Stokes flow — what breaks the symmetry?* If you ask an AI assistant the same question, treat its reply as a *claim to be tested*, not an answer: it will sound authoritative either way. Keep both answers. At the end of Chapter 1 you will have derived the drag and the pressure/viscous split yourself — come back and see which prediction survives contact with the mathematics. That is the loop every orange box runs:

predict or generate, then check against something you have derived.

Carry four things into Chapter 1: the Stokes equations (0.4)–(0.5) and their three gifts (linearity, instantaneity, reversibility); the stress tensor (0.9) and force integral (0.10), your route to drag; enough index notation to read a Stokeslet without flinching; and the reciprocal theorem (0.11), filed away for when you need it. On to the single sphere.

Chapter 1

Flow around a single sphere

This is the "hydrogen atom" of low-Reynolds-number hydrodynamics: viscous flow past a rigid sphere. We solve it in full with the axisymmetric stream function from MA3D1, harvesting four results that echo through everything that follows — the velocity and pressure fields, the **Stokes drag** $F = 6\pi\mu a U$, its split into pressure and friction parts, and the surprisingly *long* reach of the disturbance — then recognise the sphere as a sum of two elementary singularities, the building blocks of Chapters 2 and 3.

1.1 Set-up and symmetry

Take a rigid sphere of radius a held fixed at the origin, with a uniform stream $\boldsymbol{u} \to U\hat{\boldsymbol{z}}$ far away. (By Galilean-plus-Stokes reasoning — add a uniform velocity, which is itself a trivial Stokes flow — this is equivalent to a sphere translating at $-U\hat{\boldsymbol{z}}$ through fluid at rest, and we will switch freely between the two pictures.) Use spherical polars (r, θ) with θ measured from the $+z$ axis; by axisymmetry nothing depends on the azimuthal angle ϕ and $u_\phi = 0$ (Figure 1.1).

Because the flow is axisymmetric and incompressible, we may use the **Stokes stream function** $\psi(r, \theta)$, exactly as in MA3D1, with

$$u_r = \frac{1}{r^2 \sin\theta}\frac{\partial \psi}{\partial \theta}, \qquad u_\theta = -\frac{1}{r \sin\theta}\frac{\partial \psi}{\partial r}. \tag{1.1}$$

This construction satisfies $\boldsymbol{\nabla} \cdot \boldsymbol{u} = 0$ automatically.

1.2 From Stokes to the stream function

Take the curl of the Stokes momentum equation (0.4). The pressure gradient dies ($\boldsymbol{\nabla} \times \boldsymbol{\nabla} p = 0$), leaving $\boldsymbol{\nabla} \times (\nabla^2 \boldsymbol{u}) = \boldsymbol{0}$, i.e. the vorticity is harmonic. In axisymmetric flow the only vorticity component is ω_ϕ, and one shows (Exercise 1.1) that this reduces to a single scalar equation for the stream function,

$$E^2\Big(E^2 \psi\Big) = 0, \qquad E^2 := \frac{\partial^2}{\partial r^2} + \frac{\sin\theta}{r^2}\frac{\partial}{\partial \theta}\left(\frac{1}{\sin\theta}\frac{\partial}{\partial \theta}\right). \tag{1.2}$$

The operator E^2 is *not* the Laplacian: it is the operator that turns a stream function into (minus) the azimuthal vorticity, and it is the one that appears naturally when you write the axisymmetric Stokes problem in terms of ψ. Its precise form is derived in Exercise 1.1; all we need here is that (1.2) is our field equation, linear and fourth order. It is also *separable* in the following sense. The far-field condition (next section) forces the angular dependence of ψ to be exactly $\sin^2\theta$ — a single angular mode. Because E^2 acting on $f(r)\sin^2\theta$ returns something of the same form $[\,\cdot\,]\sin^2\theta$ (the $\sin^2\theta$ factors straight through, as you verify in Exercise 1.1), the whole problem collapses to a single ordinary differential equation for the radial part $f(r)$. That is what "separable" buys us here: one ODE instead of a partial differential equation.

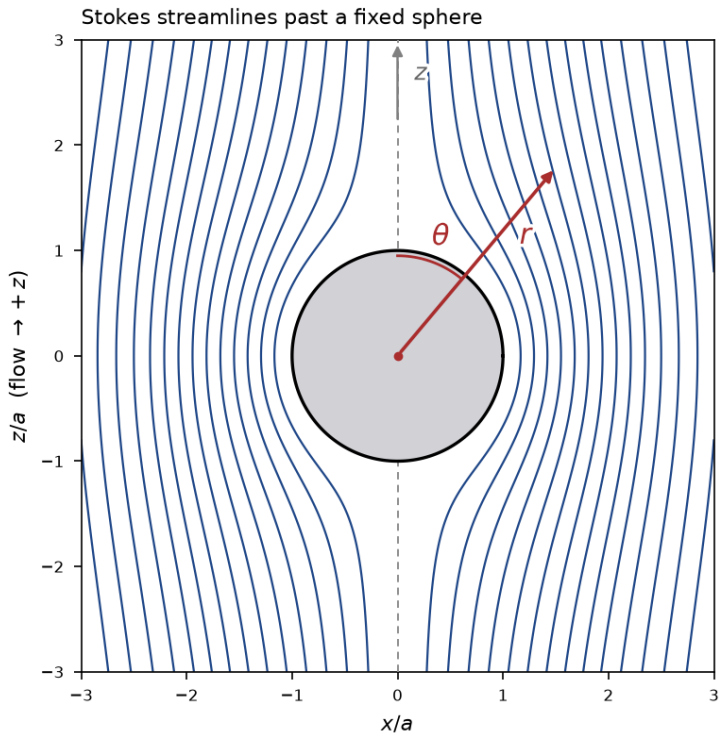


Figure 1.1: Uniform Stokes flow (speed U, along $+z$) past a fixed rigid sphere of radius a. The curves are true streamlines — contours of the stream function (1.9) — so the pattern is exactly fore–aft symmetric, and the streamlines are still visibly displaced at $r = 3a$: a first glimpse of the long $1/r$ reach (Sec. 1.6). Spherical polars (r, θ) are measured from the flow axis; the problem is axisymmetric about z.

1.3 Separable solution

The far-field condition $\boldsymbol{u} \to U\hat{\boldsymbol{z}}$ translates (Exercise 1.2) into $\psi \to \frac{1}{2}Ur^2\sin^2\theta$, so we seek a solution of the form

$$\psi(r,\theta) = f(r)\,\sin^2\theta. \tag{1.3}$$

Substituting into (1.2), the angular parts factor out and f obeys an *equidimensional* ordinary differential equation — one in which each term carries the same total power of r (so $r\,\mathrm{d}/\mathrm{d}r$ appears throughout), also called an *Euler* or *Cauchy–Euler* equation. Such an equation is solved by trying power-law solutions $f \propto r^n$; here it is fourth order, with four independent solutions, the powers r^{-1}, r, r^2, r^4 (Exercise 1.1). Hence the general admissible stream function is

$$\psi = \left(\frac{A}{r} + B\,r + C\,r^2 + D\,r^4\right)\sin^2\theta. \tag{1.4}$$

Four constants, to be fixed by four boundary conditions — two at infinity, two on the sphere.

Far field. Matching $\psi \to \frac{1}{2}Ur^2\sin^2\theta$ as $r \to \infty$ forces the fastest-growing term to vanish and sets the r^2 coefficient:

$$D = 0, \qquad C = \tfrac{1}{2}U. \tag{1.5}$$

(The r^4 term would give a velocity growing like r^2: unphysical.)

On the sphere. No-slip *and* no-penetration on a rigid, stationary sphere mean the full velocity vanishes at $r = a$: from (1.1) this is $u_r = u_\theta = 0$, equivalently

$$f(a) = 0, \qquad f'(a) = 0. \tag{1.6}$$

With $f(r) = A/r + Br + \frac{1}{2}Ur^2$, conditions (1.6) are two linear equations for A, B:

$$\frac{A}{a} + Ba + \tfrac{1}{2}Ua^2 = 0, \qquad -\frac{A}{a^2} + B + Ua = 0. \tag{1.7}$$

Solving (do it — Exercise 1.2) gives

$$A = \tfrac{1}{4}Ua^3, \qquad B = -\tfrac{3}{4}Ua. \tag{1.8}$$

We have our answer. The Stokes stream function for flow past a sphere is

$$\boxed{\psi(r,\theta) = \frac{U}{4}\left(2r^2 - 3ar + \frac{a^3}{r}\right)\sin^2\theta.} \tag{1.9}$$

1.4 The velocity and pressure fields

Differentiating (1.9) through (1.1):

$$\boxed{u_r = U\cos\theta\left(1 - \frac{3a}{2r} + \frac{a^3}{2r^3}\right),} \tag{1.10}$$

$$\boxed{u_\theta = -U\sin\theta\left(1 - \frac{3a}{4r} - \frac{a^3}{4r^3}\right).} \tag{1.11}$$

You should sanity-check the limits: at $r = a$ both vanish (no-slip); as $r \to \infty$ they return $u_r \to U\cos\theta$, $u_\theta \to -U\sin\theta$, which is precisely $U\hat{\boldsymbol{z}}$ resolved into polars.

The pressure follows by integrating the Stokes momentum equation (0.4) (Exercise 1.3):

$$\boxed{p = p_\infty - \frac{3\mu U a \cos\theta}{2r^2}.} \tag{1.12}$$

Note what *kind* of field this is: $\cos\theta/r^2$ is a **potential dipole** — exactly the pressure you would write down for a point source and sink, or equivalently the potential of a dipole aligned with $\hat{\boldsymbol{z}}$. It is high on the upstream face ($\theta = \pi$) and low on the downstream face, and — crucially — it is *fore–aft antisymmetric*. That antisymmetry is what gives a net force, in stark contrast to the fore–aft *symmetric* pressure of ideal flow that produced d'Alembert's zero drag.

A picture worth holding

The streamlines of (1.9) are perfectly fore–aft symmetric: the flow looks identical if you reverse the stream direction. This is reversibility (Chapter 0) made visible. Yet the sphere feels a drag — reversibility constrains the *kinematics* (the streamline pattern), not the *dynamics* (the force). Part III's first Python tool draws exactly this picture; you will validate your streamline plot against Equation (1.9).

1.5 The drag: three ways to $6\pi\mu aU$

(a) The surface-traction integral

The force on the sphere is the traction (0.10) integrated over its surface. Evaluating the stress tensor (0.9) on $r = a$ gives a remarkably simple result: the traction (which we denote $\boldsymbol{\tau}$, reserving $\boldsymbol{t}$ for the tangent vector) is *uniform* over the sphere and points along the flow,

$$\boldsymbol{\tau}\big|_{r=a} = \frac{3\mu U}{2a}\,\hat{\boldsymbol{z}} \qquad \text{(independent of } \theta\text{)}. \tag{1.13}$$

The net force is then trivial:

$$\boldsymbol{F} = \oint_{r=a} \boldsymbol{\tau}\,\mathrm{d}S = \frac{3\mu U}{2a}\,\hat{\boldsymbol{z}} \times 4\pi a^2 = 6\pi\mu a U\,\hat{\boldsymbol{z}}. \tag{1.14}$$

$$\boxed{\boldsymbol{F} = 6\pi\mu a\,\boldsymbol{U}} \qquad \text{(Stokes drag, 1851).} \tag{1.15}$$

Linear in U, linear in a, linear in μ: every one of those is a direct fingerprint of the linearity of the Stokes equations. Contrast the high-Re drag $\sim \rho U^2 a^2$, quadratic in speed.

(b) The pressure/friction split (the $\frac{1}{3}$–$\frac{2}{3}$ rule)

Splitting the traction (1.13) into its pressure part ($-p\boldsymbol{n}$) and its viscous part reveals a memorable division of labour (Exercise 1.4):

$$\underbrace{2\pi\mu a U}_{\text{pressure (form) drag}} + \underbrace{4\pi\mu a U}_{\text{viscous (skin-friction) drag}} = 6\pi\mu a U. \tag{1.16}$$

So *one third* of Stokes drag is pressure and *two thirds* is viscous friction — a useful number to remember, and a good check on any code that computes drag by integrating a stress field.

(c) The singularity shortcut (a preview)

Section 1.7 will show that the disturbance field is a Stokeslet of strength $6\pi\mu aU$ plus a force-free potential dipole. Since a Stokeslet of strength $\boldsymbol{F}$ exerts exactly the net force $\boldsymbol{F}$ on the fluid (and the dipole exerts none), the drag can be *read off* the far-field coefficient with no surface integral at all. This is the shortcut that makes multi-sphere problems tractable.

(d) And the torque?

Alongside the force, a body in a viscous flow also feels a *torque* about its centre,

$$\boldsymbol{T} = \oint_{r=a} \boldsymbol{x} \times \boldsymbol{\tau}\, \mathrm{d}S = \oint_{r=a} \boldsymbol{x} \times (\mathsf{T} \cdot \boldsymbol{n})\, \mathrm{d}S. \tag{1.17}$$

For our translating sphere the torque is *zero*, and it is worth seeing why: the traction (1.13) is uniform and parallel to $\hat{\boldsymbol{z}}$, so $\boldsymbol{x} \times \boldsymbol{\tau}$ integrates to zero over the sphere by fore–aft symmetry. A sphere pushed through still fluid does not spin — there is no preferred sense of rotation for it to pick. The torque becomes non-trivial only when that symmetry is broken: by a second sphere nearby (Chapter 2), by a wall, or by an ambient shear. We introduce it now so the machinery is in place when it stops vanishing.

1.6 The long reach of Stokes flow

Subtract the uniform stream to isolate the **disturbance** the sphere creates. The leading term as $r \to \infty$ is

$$\boldsymbol{u} - U\hat{\boldsymbol{z}} \;\sim\; -\frac{3a}{4r}\, U\,(\ldots) \;=\; O\!\left(\frac{a}{r}\right). \tag{1.18}$$

The disturbance decays only as $1/r$. Compare potential flow past a sphere, where the doublet disturbance decays as $1/r^3$. This slow, $1/r$ decay is the single most consequential fact in the book:

Key result 1.1. A body in Stokes flow disturbs the fluid over a *long* range, $\sim 1/r$. Boundaries therefore feel each other strongly and at large separations. The strong, long-ranged **hydrodynamic interactions** of Chapters 2–3 are a direct consequence.

Stokes' paradox: why there is no 2D version

Try the same calculation for a *cylinder* (2D flow past a circle) and you hit a wall: there is *no* solution that is both no-slip on the cylinder *and* bounded at infinity. This is **Stokes' paradox**. Physically, the $1/r$ reach is so strong that in 2D the disturbance cannot be confined — inertia, however small, must re-enter far away (Oseen's resolution). It is the low-Re mirror of the high-Re surprises you met with cylinders in MA3D1, and a healthy reminder that "creeping flow" is not always benign.

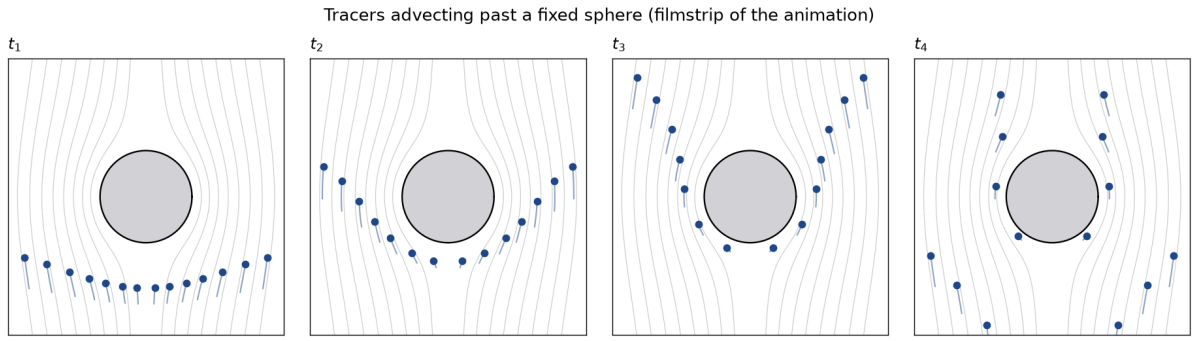


Figure 1.2: Passive tracers carried past the fixed sphere by the flow (1.9), shown at four successive times. They splay wide as they approach, slip around the sphere, and reconverge behind it on a mirror-image path: the fore–aft symmetry of Stokes flow made visible. Notice how far out ($r \gg a$) the tracers are still deflected — the long $1/r$ reach again.

1.7 The sphere as a sum of singularities

The most powerful way to view the result (1.9) is through the elementary singular solutions of the Stokes equations — the low-Re analogues of the sources and dipoles you superposed in potential flow.

The Stokeslet (Oseen tensor). Make the phrase "point force" precise: a Stokeslet is the flow driven by a force concentrated at a single point, i.e. the solution of the Stokes equations with a Dirac-delta forcing,

$$\boldsymbol{\nabla}\cdot\boldsymbol{u}=0, \qquad \mathbf{0}=-\boldsymbol{\nabla}p+\mu\nabla^2\boldsymbol{u}+\boldsymbol{F}\,\delta(\boldsymbol{x}), \tag{1.19}$$

the exact analogue of the source/dipole singularities you placed to build potential flows in MA3D1. The solution is the fundamental (Green's) function

$$\boxed{u_i(\boldsymbol{x})=\frac{1}{8\pi\mu}\,G_{ij}(\boldsymbol{x})\,F_j, \qquad G_{ij}(\boldsymbol{x})=\frac{\delta_{ij}}{r}+\frac{x_ix_j}{r^3}, \qquad p=\frac{F_jx_j}{4\pi r^3},} \tag{1.20}$$

where G_{ij} is the **Oseen tensor** (or Stokeslet). It is the Green's function of Stokes flow: the response to a δ-function force. Its $1/r$ decay is exactly the long reach we just found — the Stokeslet *is* the far field of any forced body. One point often causes worry: the singularity at $\boldsymbol{x}=\mathbf{0}$ looks alarming, but it is harmless *here* because it sits *inside* the solid sphere ($r<a$), where there is no fluid. We only ever use (1.20) in the fluid $r\geq a$, where it is perfectly smooth — exactly as a potential-flow doublet placed at a body's centre is used only outside the body.

The potential (source) dipole. The second ingredient is a force-free, faster-decaying singularity,

$$u_i^{\mathrm{D}}(\boldsymbol{x})=\left(-\frac{\delta_{ij}}{r^3}+\frac{3x_ix_j}{r^5}\right)d_j, \tag{1.21}$$

which is precisely the potential-flow doublet from MA3D1 (it is harmonic and carries no net force). It decays as $1/r^3$.

The sphere is their sum. A short calculation (Exercise 1.6) shows that the translating-sphere disturbance is

$$\boxed{u_i=\underbrace{\frac{6\pi\mu aU_j}{8\pi\mu}\left(\frac{\delta_{ij}}{r}+\frac{x_ix_j}{r^3}\right)}_{\text{Stokeslet, }\boldsymbol{F}=6\pi\mu a\boldsymbol{U}}-\underbrace{\frac{a^3}{4}U_j\left(-\frac{\delta_{ij}}{r^3}+\frac{3x_ix_j}{r^5}\right)}_{\text{potential dipole}}.} \tag{1.22}$$

Two singularities — one carrying the net force and the long range, one enforcing no-slip on the surface — reproduce the entire flow. This is the germ of the whole subject: represent bodies by singularities, and interactions by superposing them. Chapter 2 does exactly this for two spheres.

Python & AI

Build it. Write a short Python function `sphere_velocity(x, U, a)` that evaluates the Stokeslet-plus-dipole field (1.22) at any point $\boldsymbol{x}$ (an AI assistant will draft this in seconds — and will very likely get the dipole *sign* wrong).

Check it. Evaluate your function at a few points on the surface $r=a$: it must return exactly $\boldsymbol{U}$ there (no-slip). If it does not, the dipole term needs a minus sign — the strength is $-\frac{1}{4}a^3\boldsymbol{U}$. This single surface test catches the most common error in the whole construction, and you now have the analytic answer (1.9) to check against line by line.

That check — not the AI's confidence — is what tells you the code is right. (The finished version is `sphere_velocity_from_singularities` in `stokeslets.py`, Appendix C.)

1.8 Exercises

Exercise 1.1

The field equation.

(a) For axisymmetric flow, show the only non-zero vorticity component is $\omega_\phi = -\frac{1}{r\sin\theta}E^2\psi$, with E^2 as in (1.2).

(b) Hence show that taking the curl of the Stokes equations gives $E^4\psi = 0$.

(c) Substituting $\psi = f(r)\sin^2\theta$, show $E^2\psi = \left(f'' - 2f/r^2\right)\sin^2\theta$ and deduce the equidimensional (Cauchy–Euler) ODE for f — the one solved by powers r^n — whose four solutions are r^{-1}, r, r^2, r^4.

Exercise 1.2

Boundary conditions. Show that $\boldsymbol{u} \to U\hat{\boldsymbol{z}}$ corresponds to $\psi \to \frac{1}{2}Ur^2\sin^2\theta$. Then impose (1.6) and solve for A and B, confirming $A = \frac{1}{4}Ua^3$, $B = -\frac{3}{4}Ua$.

Exercise 1.3

The pressure. Using $\boldsymbol{\nabla}p = \mu\nabla^2\boldsymbol{u}$ and the velocity field, integrate to obtain (1.12). *Hint:* you only need the r- or θ-component; the other checks consistency.

Exercise 1.4

Form vs. friction. Verify (1.13) by evaluating $T_{ij}n_j$ on $r = a$, then separate the pressure and viscous contributions to the z-force and confirm the $2\pi\mu aU + 4\pi\mu aU$ split.

Exercise 1.5

Sedimentation. A sphere of density ρ_s settles under gravity through fluid of density ρ_f and viscosity μ. Balancing net weight against Stokes drag, show the terminal velocity is

$$U_{\text{sed}} = \frac{2}{9}\frac{(\rho_s - \rho_f)ga^2}{\mu}. \tag{1.23}$$

Evaluate it for a 10 µm water droplet in air ($\mu_{\text{air}} \approx 1.8\times10^{-5}\,\text{Pa s}$, $\rho_s \approx 10^3\,\text{kg m}^{-3}$) and compare with the $3\,\text{mm s}^{-1}$ quoted in Chapter 0's table. This is the settling speed whose *modification by a neighbour* is the whole subject of Chapter 2.

Exercise 1.6

Singularity decomposition. Starting from (1.22), verify (i) that it satisfies $\boldsymbol{u} = \boldsymbol{U}$ on $r = a$, and (ii) that its far field reproduces the u_r, u_θ above. Identify which term carries the net force and explain why the dipole cannot contribute to drag.

Exercise 1.7

Reversibility and a falling sphere. Argue from kinematic reversibility (Chapter 0) that a rigid sphere settling under gravity next to a vertical plane wall cannot drift towards or away from the wall at leading order in Stokes flow. *(Careful: the real story is subtler once you include the wall's induced flow — see Chapter 2. State precisely what symmetry*

argument you are using and where it might break.)

Chapter 2

Two spheres, and the forces between them

One sphere was a set-piece. *Two* spheres is where low-Reynolds-number hydrodynamics becomes a living subject — and where your project begins. The long $1/r$ reach of Chapter 1 means the spheres feel each other strongly even when far apart: two settling fog droplets do not fall independently, but each rides in the other's disturbance flow, and the pair falls faster than either alone. Computing these **hydrodynamic interactions** is what Chapter 3's tool is for.

We attack from the two natural ends. *Far apart*, the interaction is weak and expands in a/d (method of reflections), recovering the tensor mobilities of point-particle simulations. *Nearly touching*, it is violent and singular, dominated by the thin film in the gap (lubrication). Chapter 3 stitches the two together, as Rosa *et al.* do for cloud droplets.

2.1 Resistance and mobility: forces as a linear map

Everything rests on one consequence of linearity (Chapter 0). Because the Stokes equations are linear and instantaneous, the forces and torques on a collection of rigid particles are a *linear function* of their translational and angular velocities — and vice versa. For two spheres, collect the two forces and two torques into a generalised force $\mathcal{F} = (\boldsymbol{F}_1, \boldsymbol{T}_1, \boldsymbol{F}_2, \boldsymbol{T}_2)$ and the velocities into $\mathcal{U} = (\boldsymbol{U}_1, \boldsymbol{\Omega}_1, \boldsymbol{U}_2, \boldsymbol{\Omega}_2)$. Then

$$\boxed{\mathcal{F} = -\,\mathsf{R}\,\mathcal{U}, \qquad \text{equivalently} \qquad \mathcal{U} = -\,\mathsf{M}\,\mathcal{F}, \qquad \mathsf{M} = \mathsf{R}^{-1}.} \tag{2.1}$$

R is the **grand resistance matrix** (velocities in, forces out — the natural object when you *prescribe* motion), and M is the **grand mobility matrix** (forces in, velocities out — the natural object for sedimentation, where you prescribe gravity and ask how fast things move). They are inverses. For a single sphere, R collapses to the scalar $6\pi\mu a$ of Chapter 1; the whole of this chapter is the story of the *off-diagonal, sphere-1–sphere-2 blocks* that a single sphere cannot have.

Symmetry from the reciprocal theorem

The grand resistance matrix is **symmetric and positive-definite**, $\mathsf{R} = \mathsf{R}^{\mathsf{T}}$. This is not obvious — it says the force on sphere 1 caused by translating sphere 2 is tied, component by component, to the force on sphere 2 caused by translating sphere 1. It is a direct payoff of the Lorentz reciprocal theorem (0.11): apply it with the "unhatted" flow being sphere 2 in motion and the "hatted" flow sphere 1 in motion, and the equality of cross-work is exactly $R_{12} = R_{21}$. Positive-definiteness is just the statement that moving particles dissipate energy. Your Chapter 3 code should reproduce this symmetry to machine precision — a

superb built-in correctness check.

Reducing by symmetry: two identical spheres

For two identical spheres a great simplification follows from axisymmetry about the line of centres $\hat{\boldsymbol{d}}$ (Figure 2.1). Any motion decomposes into components *along* $\hat{\boldsymbol{d}}$ and *transverse* to it, and these do not mix. Writing $d = |\boldsymbol{x}_2 - \boldsymbol{x}_1|$ for the centre-to-centre distance, the translation–translation mobility relating the velocity of sphere α to a force on sphere β takes the form

$$\mathsf{M}^{\alpha\beta} = \frac{1}{6\pi\mu a}\Big[\, x^{\alpha\beta}(d)\,\hat{\boldsymbol{d}}\,\hat{\boldsymbol{d}} + y^{\alpha\beta}(d)\,(\mathsf{I} - \hat{\boldsymbol{d}}\,\hat{\boldsymbol{d}})\Big], \tag{2.2}$$

so the entire translational problem is carried by two *scalar* functions for each pair $(\alpha\beta)$: the *longitudinal* mobility $x^{\alpha\beta}$ (motion along $\hat{\boldsymbol{d}}$) and the *transverse* mobility $y^{\alpha\beta}$ (motion across it). The two tensors in (2.2) are complementary *projectors*. Acting on any vector, $\hat{\boldsymbol{d}}\,\hat{\boldsymbol{d}}$ keeps only the part *along* the line of centres (it returns $(\boldsymbol{v}\cdot\hat{\boldsymbol{d}})\,\hat{\boldsymbol{d}}$), while $\mathsf{I} - \hat{\boldsymbol{d}}\,\hat{\boldsymbol{d}}$ keeps only the part *perpendicular* to it; together they add back to the identity, $\hat{\boldsymbol{d}}\hat{\boldsymbol{d}} + (\mathsf{I} - \hat{\boldsymbol{d}}\hat{\boldsymbol{d}}) = \mathsf{I}$. So (2.2) just says: respond with mobility $x^{\alpha\beta}$ to the pushed-along-$\hat{\boldsymbol{d}}$ part of the force and with $y^{\alpha\beta}$ to the sideways part — exactly the along/across split the geometry demands. The self terms $x^{11}, y^{11} \to 1$ as $d \to \infty$ (an isolated sphere), and the cross terms $x^{12}, y^{12} \to 0$. Our whole task is to find these four functions — first for large d, then for small gaps.

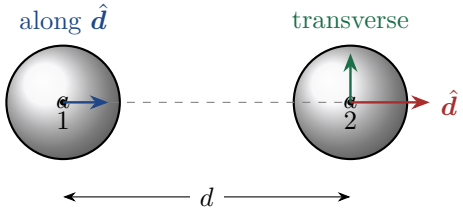


Figure 2.1: Two equal spheres at centre-to-centre separation d, with unit vector $\hat{\boldsymbol{d}}$ along the line of centres. By axisymmetry, motion splits into a *longitudinal* part (along $\hat{\boldsymbol{d}}$, carried by the scalars $x^{\alpha\beta}$) and a *transverse* part (across it, carried by $y^{\alpha\beta}$); the two do not couple.

2.2 Far field I: the method of reflections

The idea is beautifully physical and rests entirely on Chapter 1. Sphere 1 moves; by Chapter 1 it radiates a disturbance flow (a Stokeslet plus dipole). Sphere 2 finds itself bathed in this *ambient* flow and must respond. To work out its response we need one more exact result about a sphere in an imposed flow.

Faxén's laws

Key result 2.1. Faxén's laws. A rigid sphere of radius a centred at $\boldsymbol{x}_0$, translating at $\boldsymbol{U}$ and rotating at $\boldsymbol{\Omega}$ in an ambient Stokes flow $\boldsymbol{u}^\infty(\boldsymbol{x})$ (the flow that would exist in its absence), experiences

$$\boldsymbol{F} = 6\pi\mu a\left[\boldsymbol{u}^\infty(\boldsymbol{x}_0) + \frac{a^2}{6}\nabla^2\boldsymbol{u}^\infty(\boldsymbol{x}_0) - \boldsymbol{U}\right], \tag{2.3}$$

$$\boldsymbol{T} = 8\pi\mu a^3\left[\tfrac{1}{2}\boldsymbol{\nabla}\times\boldsymbol{u}^\infty(\boldsymbol{x}_0) - \boldsymbol{\Omega}\right]. \tag{2.4}$$

Read (2.3) carefully: the force is $6\pi\mu a$ times the difference between the sphere's velocity and a suitably *averaged* ambient velocity at its centre — the $a^2\nabla^2/6$ term is the finite-size correction that distinguishes a sphere from a point. (These laws are themselves a clean application of the reciprocal theorem; deriving (2.3) is Exercise 2.3.) Faxén's laws are the engine of every far-field interaction calculation and of the RPY tensor below.

Leading interaction: the Oseen reflection

Now run the reflection. Let both spheres be subject to equal external forces $\boldsymbol{F}$. The picture to hold in mind is two identical droplets settling side by side under gravity: gravity pulls each with the same downward force $\boldsymbol{F} = m'\boldsymbol{g}$ (buoyancy-corrected weight), and we ask how fast each moves and how the presence of the other changes that. This is the sedimentation problem, and it is the simplest setting in which the two spheres genuinely feel one another. A word on what $\boldsymbol{F}$ *is*: it is the external force applied to each sphere — here gravity (buoyancy-corrected weight), and nothing else. The spheres do *not* exert any direct force on one another; there is no contact, no potential between them. They interact only *hydrodynamically*: each sphere, in moving, sets the surrounding fluid in motion, and that fluid motion sweeps its neighbour along. All of that coupling is carried by the mobility matrix (through the ambient flow below), never by adding an inter-particle force to $\boldsymbol{F}$. To leading order sphere 1 acts as a Stokeslet of strength $\boldsymbol{F}$, producing at sphere 2's centre the ambient flow obtained by evaluating the Stokeslet field (1.20) at the separation vector $\boldsymbol{x}_2 - \boldsymbol{x}_1 = d\,\hat{\boldsymbol{d}}$ (so $r = d$, and $\delta_{ij}/r + x_i x_j/r^3$ becomes $(\mathsf{I} + \hat{\boldsymbol{d}}\hat{\boldsymbol{d}})/d$):

$$\boldsymbol{u}^{\infty}(\boldsymbol{x}_2) = \frac{1}{8\pi\mu d}\left(\mathsf{I} + \hat{\boldsymbol{d}}\,\hat{\boldsymbol{d}}\right)\cdot\boldsymbol{F}. \tag{2.5}$$

In words: (2.5) is simply "the flow that sphere 1's point force sends to where sphere 2 sits". Faxén's law (2.3) (with the sphere free of *net* hydrodynamic force beyond $\boldsymbol{F}$) then gives sphere 2's velocity. Carrying this out for both spheres yields the cross-mobility at leading order,

$$\boxed{x^{12} = \frac{3a}{2d} + O\!\left(\frac{a^3}{d^3}\right), \qquad y^{12} = \frac{3a}{4d} + O\!\left(\frac{a^3}{d^3}\right).} \tag{2.6}$$

The longitudinal coupling is exactly *twice* the transverse one at this order — the signature of the $(\mathsf{I} + \hat{\boldsymbol{d}}\hat{\boldsymbol{d}})$ structure of the Stokeslet. Physically: a sphere drags the fluid (and hence its neighbour) along with it more effectively in the direction it moves than to the side.

The Rotne–Prager–Yamakawa mobility

Keeping the finite-size ($a^2\nabla^2/6$) term in Faxén's law promotes the Stokeslet to the **Rotne–Prager–Yamakawa (RPY)** tensor, the workhorse mobility of Brownian- and Stokesian-dynamics simulations. For $d \geq 2a$,

$$\boxed{x^{12}_{\mathrm{RPY}} = \frac{3a}{2d} - \left(\frac{a}{d}\right)^3,} \tag{2.7}$$

$$\boxed{y^{12}_{\mathrm{RPY}} = \frac{3a}{4d} + \frac{1}{2}\left(\frac{a}{d}\right)^3.} \tag{2.8}$$

The $(a/d)^3$ terms are the dipole (finite-size) reflections. RPY has two virtues your project will care about: it is *exact* through $O(d^{-3})$, and — unlike a bare Stokeslet superposition — it stays positive-definite for all configurations with $d \geq 2a$, so a simulation built on it never produces unphysical negative dissipation. Chapter 3's Python tool implements (2.7)–(2.8) and checks them against the reflection series (2.6).

Python & AI

Build it. Ask an AI assistant to write two functions returning the RPY cross-mobilities (2.7)–(2.8) as functions of $s = d/a$. Most models produce these correctly for $d \geq 2a$.
Check it, and stress-test it. First verify the large-s limit: your functions must reduce to the leading reflection $x^{12} \to 3a/2d$, $y^{12} \to 3a/4d$ of (2.6) (drop the $(a/d)^3$ term and compare). Then probe the edge: ask the model what its formula does at $d < 2a$ (overlapping spheres). The correct RPY tensor switches to a *different*, regularised form there — a hurried answer silently reuses the $d \geq 2a$ expression out of range. This is the classic failure: confidently right in the regime it was trained on, silently wrong just outside it. Your derived limit is what exposes it. (Finished versions: `rpy_cross` in `stokeslets.py`, Appendix C.)

Exercise 2.1

Two droplets in free fall. Two identical spheres, each of net weight $\boldsymbol{F} = F\hat{\boldsymbol{d}}$ (i.e. gravity acts *along* their line of centres), settle in unbounded fluid.

(a) Using (2.1) and (2.2), show both spheres fall at the *same* speed $U_{\parallel} = \dfrac{F}{6\pi\mu a}(x^{11}+x^{12})$, so the pair translates rigidly without separating.
(b) Using the leading result (2.6), show $U_{\parallel}/U_0 = 1 + \frac{3a}{2d} + \cdots$, i.e. the pair falls *faster* than an isolated sphere.
(c) Repeat for gravity acting *transverse* to the line of centres and show $U_{\perp}/U_0 = 1 + \frac{3a}{4d} + \cdots$. Which orientation falls faster, and why does that make physical sense?
(d) Deduce that a pair released side-by-side but falling under gravity along an oblique line will *drift* — the sedimentation direction is generally not vertical. (This coupling is the seed of the notorious complexity of sedimenting suspensions.)

2.3 Far field II: what the reflection series does and does not capture

The method of reflections is an expansion in a/d. Each additional reflection adds two powers of a/d, so the series converges rapidly when the spheres are well separated (a gap of a few radii) and *slowly, then not at all* as they approach contact. The reason is precisely the long range of Chapter 1: near contact the interaction is controlled not by the smooth far field but by the thin, rapidly-varying film of fluid trapped in the gap. For that we need a different — and, pleasingly, even simpler — limit.

2.4 Near field: lubrication and the squeeze-film singularity

Bring the two spheres close, so the minimum gap

$$h = d - 2a \ll a. \tag{2.9}$$

In the crescent-shaped region near contact the two surfaces are almost parallel and almost flat, separated by a gap that varies slowly along the surface. This is the regime of **lubrication theory** — the same thin-film analysis that underlies journal bearings. Because the near-contact geometry is axisymmetric about the line of centres, it is naturally described in *cylindrical polar coordinates* (ρ, ϕ, z): z along the line of centres, ρ the in-plane distance from it (the gap radius), and ϕ the azimuthal angle, on which nothing depends. We use (ρ, ϕ, z) for the lubrication analysis and Figure 2.2 below.

Squeeze flow: normal approach

Suppose the spheres approach one another along their line of centres with relative speed $W = -\dot{h}$. Fluid must be squeezed radially out of the gap (Figure 2.2).

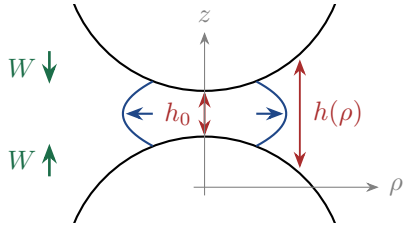


Figure 2.2: The thin lubrication gap between two near-touching spheres (minimum gap $h \ll a$). As the surfaces approach at relative speed W, fluid is squeezed radially out through the gap; the resistance to this diverges as $1/h$.

Because the gap is thin, the flow there is a nearly-unidirectional pressure-driven (Poiseuille) film, and a classical lubrication calculation (Exercise 2.2) gives the resulting force resisting approach. For two equal spheres the leading singular result is

$$F_{\text{lub}} \sim -\frac{3\pi\mu a^2}{2}\frac{W}{h} = -6\pi\mu a W \cdot \frac{1}{4}\left(\frac{a}{h}\right), \qquad h \to 0. \tag{2.10}$$

The force *diverges* as $1/h$: it becomes infinitely hard to push the last film of fluid out of the gap. Two smooth spheres in a viscous fluid can therefore never quite touch in finite time under a finite force — a fact with real consequences for droplet coalescence and particle collisions.

Writing $\xi = h/a$ for the dimensionless gap, (2.10) says the longitudinal resistance function behaves as

$$X_{11}^{A}(\xi) \sim \frac{1}{4\xi} \qquad (\xi \to 0), \tag{2.11}$$

where X_{11}^{A} is the standard normalisation (resistance divided by $6\pi\mu a$) tabulated by Jeffrey & Onishi. This is the single most important number in Chapter 3's near-field model.

Shearing motion: a gentler singularity

If instead the spheres slide *past* each other (transverse relative motion), fluid need not be expelled from the gap — it can flow around — and the singularity is much weaker: the transverse resistance diverges only *logarithmically*,

$$Y_{11}^{A}(\xi) \sim \tfrac{1}{6}\ln\frac{1}{\xi} + \text{const} \qquad (\xi \to 0). \tag{2.12}$$

(These leading near-contact coefficients — $\frac{1}{4}$ for the normal $1/\xi$ term, $\frac{1}{6}$ for the tangential log — are the equal-sphere values of the resistance functions tabulated by Jeffrey & Onishi; Chapter 3's code reproduces them numerically.) The contrast between the strong $1/\xi$ (normal) and weak $\ln(1/\xi)$ (tangential) singularities is a recurring theme of near-contact hydrodynamics and one your code should reproduce.

Key result 2.2. The two ends. Far apart, resistance functions approach their isolated-sphere values with algebraic a/d corrections (method of reflections, RPY). Near contact, the *normal* resistance diverges as $1/\xi$ and the *tangential* as $\ln(1/\xi)$ (lubrication). Chapter 3's oracle is built by combining an accurate far-field (multipole) truncation with these near-field laws — the strategy of Rosa *et al.* (2011).

Exercise 2.2

Derive the squeeze-film law. First convince yourself the gap is *locally parabolic*: a sphere surface near its south pole is $z = -\sqrt{a^2 - \rho^2} \approx -a + \rho^2/2a$ for $\rho \ll a$, so measuring height from the pole each surface rises like $\rho^2/2a$. Adding the two surfaces, the gap between two equal spheres is $h(\rho) \approx h_0 + \rho^2/a$ — a paraboloid, not a wedge. This is why a single reduced radius controls the whole calculation. Model the near-contact region as an axisymmetric gap of thickness $h(\rho) \approx h_0 + \rho^2/(2a_*)$, where ρ is the in-plane radius from the axis and $a_* = a/2$ is the reduced radius of two equal spheres (so $\rho^2/(2a_*) = \rho^2/a$, matching the paraboloid above). Assume a lubrication (Poiseuille) profile across the gap, integrate continuity to get the radial flux, and solve for the pressure $p(\rho)$; then integrate the pressure over the gap to obtain the axial force. Show you recover the $1/h_0$ scaling of (2.10), and identify the numerical coefficient. *(You will validate this coefficient numerically in Chapter 3.)*

Exercise 2.3

Faxén via reciprocity (harder, optional). Use the reciprocal theorem (0.11) with two flows around the *same* sphere — the unknown flow ($\boldsymbol{u}^\infty$, sphere held fixed) and the known translating-sphere solution of Chapter 1 — to derive the force law (2.3), including the $a^2\nabla^2/6$ term. This is the calculation that made the reciprocal theorem famous; take your time.

Exercise 2.4

A symmetry you can check. Argue from the reciprocal theorem that $x^{12} = x^{21}$ and $y^{12} = y^{21}$, and that the grand mobility matrix is symmetric. In Chapter 3 you will verify this numerically; state the tolerance you would accept as "machine precision" for a double-precision computation.

In short: two identical spheres reduce to the scalar functions $x^{\alpha\beta}, y^{\alpha\beta}$; the method of reflections and the RPY mobility (2.7)–(2.8) cover the far field, and lubrication gives the near field (normal $\sim 1/\xi$, tangential $\sim \ln(1/\xi)$). Chapter 3 blends the two into a practical oracle.

Chapter 3

A practical oracle: the Rosa method

Chapters 1 and 2 gave you the two ends of the two-sphere problem: an accurate *far-field* expansion (method of reflections / RPY) and a singular *near-field* law (lubrication). Neither alone is enough for a real simulation of settling droplets, which must handle *all* separations as particles drift together and apart. This chapter builds the bridge — a single tool, accurate everywhere — and it is the piece your project code will lean on most heavily.

This blended far-field/near-field strategy is *one of several* established ways to compute two-sphere interactions at all separations. Others include Stokesian dynamics (Brady & Bossis), full bispherical-coordinate series solutions (exact but expensive), and boundary-integral methods. We follow the Rosa approach here because of one practical advantage: by matching an accurate far-field expansion directly to the analytical lubrication singularity, it *avoids ever having to compute the awkward intermediate region* in detail — the region where neither expansion is cheap and both are needed. The strategy is that of Rosa, Wang, Maxey & Grabowski (*J. Comput. Phys.* **230**, 2011), developed to compute cloud-droplet interactions accurately and efficiently: combine an accurate multipole (force–torque–stresslet) truncation of the far field with analytical lubrication corrections in the near field. Our student version, `two_sphere_rosa.py`, keeps the essential idea but restricts to the translational problem for two equal spheres, so every line checks against Chapters 1–2. Concretely, write the pair resistance as a function of the dimensionless separation $s = d/a$ (gap $\xi = s - 2$): *far apart*, invert the RPY pair mobility of Chapter 2 (correct through $O(s^{-6})$); *near contact*, add the lubrication singularities; and blend the two with a smooth switch so neither region double-counts the other. The result is accurate from contact to infinity.

What this oracle does — and does not — cover

It is worth being explicit about the restriction, because it is exactly what makes the tool checkable. The full Rosa method handles *many* particles, each able to translate *and* rotate, coupled through a hierarchy of resistance functions (X^A, Y^A for force–translation, Y^B for translation–rotation, X^C, Y^C for torque–rotation, and stresslet terms). `two_sphere_rosa.py` keeps only the smallest piece that still shows the physics: **two equal spheres, translation only** — the functions X^A and Y^A. No rotation, no stresslet, no unequal sizes. That piece is small enough that every line checks against Chapters 1–2, and (as the next section shows) it is already enough to do real sedimentation. Rotation, unequal radii, and the Y^B coupling are the natural extensions — see the exercises.

3.1 The two ingredients

Far field (invert the mobility). For each component — motion along the line of centres ("normal", giving X^A) and transverse ("tangential", giving Y^A) — assemble the 2×2 pair

mobility from the RPY scalars and invert:

$$\mathsf{R}^{\mathrm{far}}(s) = \left[\mathsf{M}^{\mathrm{RPY}}(s)\right]^{-1}, \qquad \mathsf{M}^{\mathrm{RPY}} = \begin{pmatrix} 1 & m_{12}(s) \\ m_{12}(s) & 1 \end{pmatrix}, \tag{3.1}$$

with $m_{12} = x^{12}$ or y^{12} from Eqs. (2.7)–(2.8). This is smooth, positive-definite, and correct at large s; but it misses the divergence at contact because RPY truncates the multipole series.

Near field (add the singularity). From Chapter 2, the self resistance functions behave near contact as

$$X_{11}^{A}(\xi) \sim \frac{1}{4\xi} + \frac{9}{40}\ln\frac{1}{\xi} + O(1), \tag{3.2}$$

$$Y_{11}^{A}(\xi) \sim \frac{1}{6}\ln\frac{1}{\xi} + O(1), \tag{3.3}$$

the equal-sphere lubrication expansions of Jeffrey & Onishi (*J. Fluid Mech.* **139**, 1984). We add the *singular* part (the $1/\xi$ and ln terms) to the far-field value, letting the smooth blend absorb the finite $O(1)$ constant.

The blend. A logistic weight $w(\xi)$ in $\ln\xi$ switches from $w = 1$ (pure far field, large gap) to $w = 0$ (far field + full lubrication, small gap):

$$R(s) = R^{\mathrm{far}}(s) + \left(1 - w(\xi)\right) R^{\mathrm{lub}}_{\mathrm{sing}}(\xi). \tag{3.4}$$

Simple, transparent, and — as the validation below shows — accurate.

3.2 What it produces

Figure 3.1 shows the two resistance functions across the whole range of separations, produced by `two_sphere_rosa.py`.

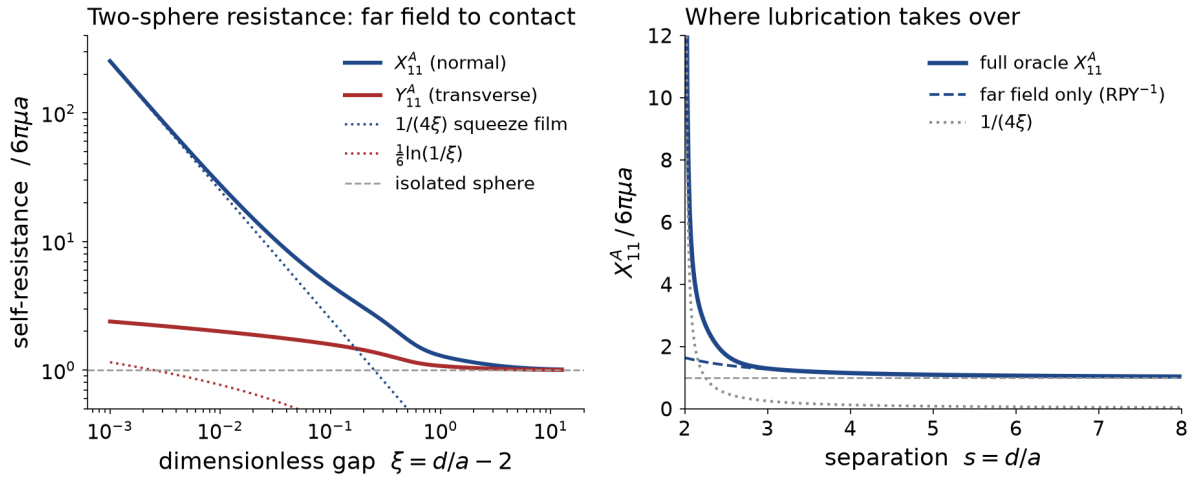


Figure 3.1: Two-sphere translational resistance functions from the Rosa-style oracle. **Left:** X_{11}^{A} (normal) diverges as the squeeze-film $1/(4\xi)$ at contact and decays to the isolated-sphere value 1 far away; Y_{11}^{A} (transverse) shows only the mild $\frac{1}{6}\ln(1/\xi)$ divergence. **Right:** the full oracle versus the far-field-only (RPY-inverse) value, showing where the lubrication correction takes over.

3.3 A worked example: two droplets sedimenting

The oracle earns its keep the moment you point it at a physical question. Take the sedimentation problem of Chapter 2 — two equal droplets, each pulled by the same gravitational force $\boldsymbol{F}$ — and ask how fast the pair falls. This is a *mobility* problem (force known, velocity sought), so we use the pair mobility $\mathsf{M} = \mathsf{R}^{-1}$ directly: the velocity of either sphere is $U = (m_{11} + m_{12})F$, i.e. its own mobility plus the drag-along from its partner. In the units where an isolated sphere settles at $U_0 = 1$:

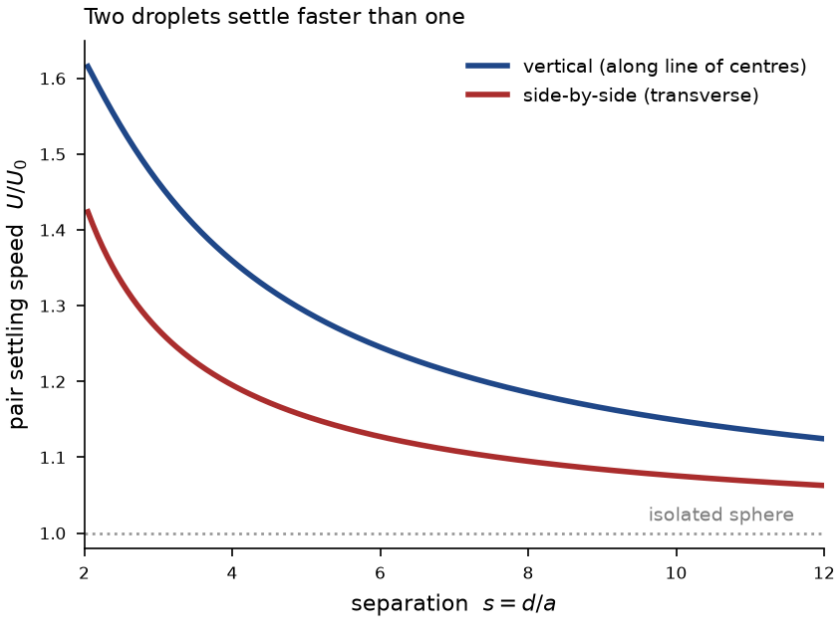


Figure 3.2: Two equal droplets settling under gravity fall *faster* than a lone droplet, because each rides in the downward flow the other creates. The effect is stronger when they are aligned vertically (along the line of centres) than side by side, by roughly the factor-of-two anisotropy of the cross-mobility (Sec. 2.2). Both curves decay to the isolated-sphere speed as the separation grows. Produced by `two_sphere_rosa.py`.

Two physical facts fall straight out, and both are checkable against Chapter 2. First, a pair *always* settles faster than a single drop (the cross-mobility is positive: each drop sits in the other's downwash). Second, a vertically-aligned pair settles faster than a side-by-side pair — at $s = 4$, for instance, $U/U_0 = 1.36$ versus 1.20 — because the longitudinal coupling $x^{12} = 3a/2d$ is twice the transverse $y^{12} = 3a/4d$. A pair released obliquely therefore also drifts sideways as it falls: the seed of the collision problem in Exercise 3.1.

A word on inertia. When we integrate a trajectory by reading off the velocity from the applied force — $\boldsymbol{U} = \mathsf{M}\boldsymbol{F}$, then $\dot{\boldsymbol{x}} = \boldsymbol{U}$ — we are quietly neglecting the *particle's* own inertia. This is consistent with everything else in the booklet. The particle Reynolds number is small, and more to the point the viscous relaxation time $\tau_p \sim \rho_p a^2/\mu$ over which a droplet forgets its velocity is minuscule compared with the time it takes to move a distance of order its own radius. The drop therefore reaches its terminal (force-balanced) velocity almost instantly: at every instant the hydrodynamic drag balances gravity, with no $m\ddot{\boldsymbol{x}}$ term to keep. So the mobility relation *is* the equation of motion — velocity is set by the instantaneous force, not by accumulated momentum. This is the "instantaneity" gift of Chapter 0 wearing its practical hat, and it is what makes a first-order-in-time trajectory integrator (rather than Newton's second law) the right tool.

3.4 Validation: the whole point

An oracle is only as good as the checks you can put it through. The module's self-test verifies three things, each tied to a result you derived by hand:

1. **Far limit.** $X_{11}^A, Y_{11}^A \to 1$ as $s \to \infty$ — the isolated sphere of Chapter 1. (Numerically: $X^A = 1.006$ at $s = 20$, 1.0002 at $s = 100$.)
2. **Near limit.** $X_{11}^A \cdot 4\xi \to 1$ as $\xi \to 0$ — the squeeze-film law of Chapter 2. More stringently, X_{11}^A minus the two-term Jeffrey–Onishi singular part $\left[\frac{1}{4\xi} + \frac{9}{40}\ln\frac{1}{\xi}\right]$ approaches a *finite constant* (≈ 1.64) as $\xi \to 0$, confirming that *both* singular terms are captured, not just the leading one.
3. **Symmetry.** The pair resistance matrix is symmetric to machine precision — the reciprocal theorem of Chapter 0.

Key result 3.1. This is the validation habit in miniature. Every number the oracle produces is pinned to an analytical result you can derive independently. When you build a larger interaction solver, you validate it the same way: against limits you understand, not against another black box.

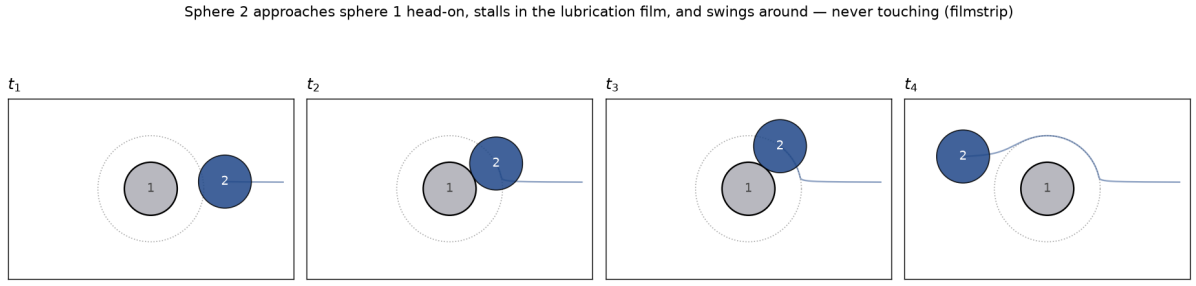


Figure 3.3: What the resistance functions of this chapter actually *do*, shown at four successive times: sphere 2 is pushed head-on toward sphere 1, but the normal resistance $X^A \sim 1/4\xi$ diverges as the gap closes, so the head-on approach stalls at the contact circle (dotted); the weaker transverse resistance $Y^A \sim \frac{1}{6}\ln(1/\xi)$ then lets it *swing around*. Two smooth spheres in Stokes flow never quite touch — exactly the $1/h$ singularity of Sec. 2.4, now driving a trajectory.

Python & AI

Build it (project-scale). Ask an AI assistant to *extend* `two_sphere_rosa.py` (Appendix C) beyond its stated scope — to the rotation–translation coupling Y^B, or to *unequal* spheres (size ratio $\beta \neq 1$). It will happily generate hundreds of lines of plausible resistance-function code.

Check it with the *same three tests* the module already uses (Sec. 3.3): does the new function reduce to the isolated-sphere limit far apart? Does it show the correct near-contact divergence? Is the full resistance matrix still symmetric (a requirement of the reciprocal theorem, Sec. 0.5)? Wire these as assertions into `run_all_tests.py` *before* you trust a single number. You are the oracle that catches the line where the generated code went wrong — which is the entire skill this booklet has been building.

Exercise 3.1

Towards collision efficiency. Two equal droplets settle under gravity, released with a horizontal offset. Using the resistance functions X_{11}^A, Y_{11}^A and the fact that gravity provides a known force, set up (you need not fully solve) the pair of ODEs for their relative trajectory. Explain physically why the $1/\xi$ divergence of X^A means that, in the Stokes approximation,

two *smooth* droplets approaching head-on never quite collide in finite time — and why this makes the modelling of rain formation subtle. *(This is the physical question Rosa et al. were built to answer.)*

Exercise 3.2

Test the blend. The choice of switch location and width in the blend is not unique. Using `two_sphere_rosa.py`, vary the blend parameters and plot X_{11}^{A}; over what range of gaps does the answer change by more than 1%? What does this tell you about the honest accuracy of a matched-asymptotic oracle in the crossover region, and how would you reduce that uncertainty (hint: more multipoles in the far field, more terms in the lubrication expansion)?

Appendix A

Solutions and hints

These are worked outlines, not full write-ups — enough to check your answer and unstick you, but leaving the algebra for you to enjoy. Exercises are grouped by chapter.

Chapter 0

Ex. 0.1 (index notation). (a) $\partial r/\partial x_i = x_i/r$ follows from $r^2 = x_k x_k$, so $2r\,\partial r/\partial x_i = 2x_i$. (b) With $T_{ij} = -p\delta_{ij} + \mu(\partial_i u_j + \partial_j u_i)$ and $\partial_k u_k = 0$, $\partial_j T_{ij} = -\partial_i p + \mu\partial_j\partial_j u_i = -\partial_i p + \mu\nabla^2 u_i$, which is 0 by the Stokes equations — every fluid element is in mechanical equilibrium. (c) Taking the antisymmetric part and using $\partial_j T_{ij} = 0$ gives $\varepsilon_{ijk} x_j \partial_l T_{kl} = 0$, the statement that torques balance too.

Chapter 1

Ex. 1.1 (E^2 operator). Take the curl of $\mathbf{0} = -\boldsymbol{\nabla} p + \mu\nabla^2\boldsymbol{u}$ to kill the pressure: $\nabla^2\boldsymbol{\omega} = \mathbf{0}$. In axisymmetric flow $\boldsymbol{\omega} = \omega_\phi\,\hat{\boldsymbol{\phi}}$ and $\omega_\phi = -E^2\psi/(r\sin\theta)$; substituting and simplifying gives $E^2(E^2\psi) = 0$ with the operator quoted in the text. The sign in the angular part is what distinguishes E^2 from ∇^2.

Ex. 1.2 (boundary conditions). No-slip and no-penetration on $r = a$ ($u_r = u_\theta = 0$) plus $\boldsymbol{u} \to U\hat{\boldsymbol{z}}$ at infinity. Writing the far field as $\psi \to \frac{1}{2}Ur^2\sin^2\theta$ fixes the r^2 coefficient; the two surface conditions fix the $1/r$ and r coefficients; the r^4 term must vanish for boundedness. Result: $\psi = \frac{U}{4}(2r^2 - 3ar + a^3/r)\sin^2\theta$.

Ex. 1.3 (pressure). From the r-momentum equation with the known $\boldsymbol{u}$, integrate $\partial p/\partial r$ to get $p = p_\infty - \frac{3}{2}\mu U a\cos\theta/r^2$. It is a pure dipole: high pressure on the upstream face, low on the downstream, decaying as $1/r^2$.

Ex. 1.4 (drag split). Integrate the pressure (form drag) and the viscous traction (friction drag) separately over the sphere. Pressure contributes $2\pi\mu aU$, viscous $4\pi\mu aU$; together $6\pi\mu aU$. The 1/3–2/3 split is a famous feature of the sphere.

Ex. 1.5 (sedimentation). Balance weight against drag: $\frac{4}{3}\pi a^3(\rho_s - \rho_f)g = 6\pi\mu a U_{\text{sed}}$, so $U_{\text{sed}} = \frac{2}{9}(\rho_s - \rho_f)ga^2/\mu$. Note the a^2: small drops settle slowly, which is why fog hangs.

Ex. 1.6 (singularity decomposition). Match the translating-sphere disturbance to a Stokeslet of strength $6\pi\mu a\boldsymbol{U}$ plus a potential dipole. Enforcing $\boldsymbol{u} = \boldsymbol{U}$ on $r = a$ fixes the dipole strength to $a^3/4$ and its sign to *minus* (Eq. (1.22)). Verify numerically with `stokeslets.py`, whose reconstruction test agrees to machine precision.

Ex. 1.7 (reversibility). Under $\boldsymbol{U} \to -\boldsymbol{U}$ the whole Stokes solution reverses (linearity), so the drag reverses direction but keeps its magnitude; a sphere settling under gravity cannot drift sideways, because that would break the fore–aft symmetry the equations respect.

Chapter 2

Ex. 2.1 (two settling droplets). (a) By symmetry both spheres share the velocity; the mobility relation gives $U_\parallel = \frac{F}{6\pi\mu a}(x^{11}+x^{12})$. (b) With $x^{11} = 1$, $x^{12} = 3a/2d$: $U_\parallel/U_0 = 1+3a/2d$. (c) Transverse uses y: $U_\perp/U_0 = 1 + 3a/4d$; the along-the-line orientation falls faster because a sphere drags fluid more effectively in its direction of motion. (d) Because the two response coefficients differ, a force oblique to the line of centres produces a velocity not parallel to it — the pair drifts sideways as it settles.

Ex. 2.2 (squeeze-film law). With gap $h(\rho) = h_0 + \rho^2/a_*$, $a_* = a/2$, a Poiseuille profile gives radial flux $q = -\frac{h^3}{12\mu}\, dp/d\rho$. Continuity with the approach speed W yields $\frac{1}{\rho}\frac{d}{d\rho}(\rho q) = W$; solve for $p(\rho)$, then $F = \int_0^\infty p\, 2\pi\rho\, d\rho$. Carrying out the integrals gives $F \sim -\frac{3\pi\mu a^2}{2}\, W/h_0$, i.e. the $1/h_0$ divergence with coefficient $3\pi/2$ (equivalently $X_{11}^A \sim 1/4\xi$).

Ex. 2.3 (Faxén via reciprocity). Pair the unknown ambient flow (sphere held fixed) with the known translating solution of Chapter 1 in the reciprocal theorem. The surface integrals collapse to $\boldsymbol{F} = 6\pi\mu a[\boldsymbol{u}^\infty(\boldsymbol{x}_0) + \frac{a^2}{6}\nabla^2\boldsymbol{u}^\infty - \boldsymbol{U}]$; the $a^2\nabla^2/6$ term is the finite-size average that emerges from integrating $\boldsymbol{u}^\infty$ over the sphere.

Ex. 2.4 (mobility symmetry). Apply the reciprocal theorem with flow 1 = sphere 1 moving, flow 2 = sphere 2 moving; the equality of cross-work gives $M_{12} = M_{21}$, hence $x^{12} = x^{21}$, $y^{12} = y^{21}$. For double precision, "machine precision" means a symmetry residual $\lesssim 10^{-14}$ relative to the matrix norm.

Chapter 3

Ex. 3.1 (towards collision efficiency). Gravity gives each droplet a known force; the relative velocity is $\dot{\boldsymbol{r}} = (\mathsf{M}_{11} - \mathsf{M}_{12}) \cdot (\boldsymbol{F}_1 - \boldsymbol{F}_2)$ built from X^A, Y^A. Along the line of centres the normal resistance $\sim 1/\xi$ means the approach speed $\dot{\xi} \propto \xi$ as $\xi \to 0$: integrating, $\xi(t) \sim \xi_0 e^{-t/\tau}$ reaches contact only as $t \to \infty$. Two smooth spheres never touch in finite time in the Stokes approximation — so rain formation must invoke something beyond pure Stokes drag (surface roughness, van der Waals, inertia, turbulence).

Ex. 3.2 (test the blend). Varying the blend switch/width changes X^A noticeably only in the crossover $\xi \sim 0.1$–1, typically by a few percent; far and near limits are pinned by analysis and do not move. This quantifies the honest uncertainty of a matched-asymptotic oracle in the crossover, and points to the two systematic improvements: more multipoles (extends the far field inward) and more lubrication terms (extends the near field outward), shrinking the gap between them.

Appendix B

Further reading

A short, deliberately curated list — enough to go deeper on every topic in this book without drowning. Full texts first, then the specific papers behind Chapters 2 and 3.

Textbooks

- **S. Kim & S.J. Karrila**, *Microhydrodynamics: Principles and Selected Applications*. Butterworth–Heinemann, 1991 (Dover reprint 2005). The standard reference for everything in Chapters 1–3: singularities, Faxén's laws, resistance and mobility, the reciprocal theorem. If you buy one book, buy this one.
- **J. Happel & H. Brenner**, *Low Reynolds Number Hydrodynamics*. Prentice–Hall, 1965 (Springer reprint). The classic; encyclopaedic on sphere problems.
- **E. Guyon, J.-P. Hulin, L. Petit & C.D. Mitescu**, *Physical Hydrodynamics*, 2nd ed. Oxford, 2015. Excellent physical intuition for the low-Reynolds-number world; a gentle companion to the above.
- **E.M. Purcell**, "Life at low Reynolds number", *Am. J. Phys.* **45** (1977) 3–11. Not a textbook but essential and delightful — the paper that named the regime this booklet lives in.

Papers behind the two-sphere chapters

- **D.J. Jeffrey & Y. Onishi**, "Calculation of the resistance and mobility functions for two unequal rigid spheres in low-Reynolds-number flow", *J. Fluid Mech.* **139** (1984) 261–290. The source of the resistance functions X^A, Y^A and their near-field lubrication expansions used in Chapters 2–3.
- **J. Rotne & S. Prager**, "Variational treatment of hydrodynamic interaction in polymers", *J. Chem. Phys.* **50** (1969) 4831–4837; and **H. Yamakawa**, *J. Chem. Phys.* **53** (1970) 436. The origin of the RPY mobility tensor of Chapter 2.
- **B. Rosa, L.-P. Wang, M.R. Maxey & W.W. Grabowski**, "An accurate and efficient method for treating aerodynamic interactions of cloud droplets", *J. Comput. Phys.* **230** (2011) 8109–8133. The method behind the Chapter 3 oracle: accurate far-field multipole truncation matched to near-field lubrication, developed for settling cloud droplets.
- **L. Durlofsky, J.F. Brady & G. Bossis**, "Dynamic simulation of hydrodynamically interacting particles", *J. Fluid Mech.* **180** (1987) 21–49. Stokesian Dynamics — the framework

that combines far-field mobility and near-field lubrication for many particles; the natural next step after this book.

Note on the Chapter 3 method. The oracle in `two_sphere_rosa.py` is a compact, student-scale realisation of the Rosa *et al.* (2011) strategy, restricted to two equal spheres and the translational resistance functions. The full method handles many droplets, rotations, and the stresslet; Jeffrey & Onishi (1984) provide the exact resistance functions it is validated against.

Appendix C

The Python toolkit, in full

This appendix collects the complete source of the three oracle modules and the test harness, so the booklet is self-contained. You do *not* need to read these listings linearly — the **Python & AI** boxes throughout the chapters build the code up piece by piece, each next to the mathematics it implements. Come here when you want the whole of a module in one place, or to copy a function as a validation reference for your own code.

All four files also ship alongside this booklet as runnable `.py` files. Running `python run_all_tests.py` executes every check tied to a booklet result; a clean run prints `10/10 checks passed`. The companion note `using_ai_assistants.md` explains the workflow these files are built around.

C.1 `sphere_single.py` — the single-sphere oracle (Chapter 1)

```
"""
sphere_single.py
================
The single-sphere Stokes-flow *oracle*.

This module implements the exact analytical solution for uniform Stokes flow
past a rigid sphere (Chapter 1 of the reading booklet), and a numerical drag
calculation by surface-traction integration.  It is deliberately written to
    be
*read*: every formula corresponds to a boxed equation in the booklet, and the
docstrings point back to it.

The point of this file is not the physics you already derived by hand -- it
    is
to give you a trusted reference.  When you (or an AI assistant) later write a
Stokes-flow solver, you validate it by comparing against the functions here,
whose every line you can check against Chapter 1.

Conventions
-----------
* Sphere of radius ``a`` centred at the origin.
* Uniform stream ``U`` in the +z direction far away; sphere held fixed.
  (Equivalently a sphere translating at ``-U z_hat``; add a uniform flow.)
* Spherical polar angle ``theta`` measured from the +z axis.
* Fluid viscosity ``mu``.  Pressure gauge ``p_inf = 0``.

All results are exact (Stokes 1851); see booklet Eqs. (1.3)-(1.7).
"""

from __future__ import annotations
import numpy as np

__all__ = [
```

```
    "stream_function", "velocity_spherical", "velocity_cartesian",
    "pressure", "traction_on_sphere", "stokes_drag_analytic",
    "drag_by_surface_integral",
]


# ---------------------------------------------------------------------------
#  Exact fields (Chapter 1)                                                 #
# ---------------------------------------------------------------------------
def stream_function(r, theta, a=1.0, U=1.0):
    r"""Stokes stream function  psi(r, theta).   Booklet Eq. (1.3):

        psi = (U/4) (2 r^2 - 3 a r + a^3/r) sin^2(theta).
    """
    r = np.asarray(r, dtype=float)
    return (U / 4.0) * (2 * r**2 - 3 * a * r + a**3 / r) * np.sin(theta)**2


def velocity_spherical(r, theta, a=1.0, U=1.0):
    r"""Velocity components (u_r, u_theta).   Booklet Eq. (1.4):

        u_r     =  U cos(theta) (1 - 3a/2r + a^3/2r^3)
        u_theta = -U sin(theta) (1 - 3a/4r - a^3/4r^3).
    """
    r = np.asarray(r, dtype=float)
    ur = U * np.cos(theta) * (1 - 1.5 * a / r + 0.5 * a**3 / r**3)
    ut = -U * np.sin(theta) * (1 - 0.75 * a / r - 0.25 * a**3 / r**3)
    return ur, ut


def velocity_cartesian(x, y, z, a=1.0, U=1.0):
    """Velocity (ux, uy, uz) on a Cartesian grid, flow along +z.

    Points inside the sphere (r < a) are returned as NaN so that plotting
    routines mask them cleanly.
    """
    x, y, z = np.broadcast_arrays(np.asarray(x, float),
                                  np.asarray(y, float),
                                  np.asarray(z, float))
    r = np.sqrt(x**2 + y**2 + z**2)
    theta = np.arccos(np.clip(z / np.where(r == 0, np.nan, r), -1, 1))
    phi = np.arctan2(y, x)
    ur, ut = velocity_spherical(r, theta, a, U)
    # spherical -> cartesian (axisymmetric about z, so u_phi = 0)
    st, ct, sp, cp = np.sin(theta), np.cos(theta), np.sin(phi), np.cos(phi)
    ux = ur * st * cp + ut * ct * cp
    uy = ur * st * sp + ut * ct * sp
    uz = ur * ct - ut * st
    inside = r < a
    ux = np.where(inside, np.nan, ux)
    uy = np.where(inside, np.nan, uy)
    uz = np.where(inside, np.nan, uz)
    return ux, uy, uz


def pressure(r, theta, a=1.0, U=1.0, mu=1.0, p_inf=0.0):
    r"""Pressure field.   Booklet Eq. (1.5):

        p = p_inf - (3 mu U a / 2) cos(theta) / r^2.
    """
```

```
92      r = np.asarray(r, dtype=float)
93      return p_inf - 1.5 * mu * U * a * np.cos(theta) / r**2
94
95
96  # ---------------------------------------------------------------------------
        #
97  #  Drag: analytic and by numerical surface integration
                                #
98  # ---------------------------------------------------------------------------
        #
99  def stokes_drag_analytic(a=1.0, U=1.0, mu=1.0):
100     r"""Stokes drag magnitude.   Booklet Eq. (1.9):   F = 6 pi mu a U."""
101     return 6.0 * np.pi * mu * a * U
102
103
104 def traction_on_sphere(theta, a=1.0, U=1.0, mu=1.0):
105     r"""Traction vector t = sigma . n on the sphere surface r = a.
106
107     A remarkable feature of the single-sphere solution (booklet Eq. (1.8)) is
108     that the traction is *uniform* over the sphere and points along the flow:
109
110         t = (3 mu U / 2a) z_hat        (independent of theta).
111
112     We nonetheless return it as a function of ``theta`` (as (t_x, t_y, t_z)
113     with the flow along z and evaluated in the x-z plane) so that a student
            can
114     integrate it numerically and *discover* the uniformity, rather than
            assume
115     it.  This is the honest way to validate a traction routine.
116     """
117     theta = np.asarray(theta, dtype=float)
118     tz = np.full_like(theta, 1.5 * mu * U / a)
119     tx = np.zeros_like(theta)
120     return tx, tz
121
122
123 def drag_by_surface_integral(n_theta=200, a=1.0, U=1.0, mu=1.0):
124     r"""Numerically integrate the surface traction to recover the drag.
125
126     Computes  F_z = \oint_S t_z dS  over the sphere by Gauss-Legendre
127     quadrature in ``theta`` (the phi-integral gives a factor 2 pi by
128     axisymmetry).  Returns the numerical drag; compare with
129     ``stokes_drag_analytic``.
130
131     This is the prototype for how *any* Stokes-flow code extracts a force
            from
132     a computed stress field, so getting it to converge to 6 pi mu a U here is
133     the first validation milestone.
134     """
135     # Gauss-Legendre nodes/weights in cos(theta) = mu_c on [-1, 1]
136     mu_c, w = np.polynomial.legendre.leggauss(n_theta)
137     theta = np.arccos(mu_c)
138     _, tz = traction_on_sphere(theta, a, U, mu)
139     # dS = a^2 sin(theta) dtheta dphi ;  integral over phi = 2 pi
140     # with substitution cos(theta)=mu_c, sin(theta) dtheta = -d(mu_c),
141     # so \oint t_z dS = 2 pi a^2 \int_{-1}^{1} t_z d(mu_c).
142     Fz = 2 * np.pi * a**2 * np.sum(w * tz)
143     return Fz
144
145
146 def drag_by_midpoint_rule(n_theta=50, a=1.0, U=1.0, mu=1.0):
147     r"""Same surface integral, but with a *naive* uniform midpoint rule.
148
```

```
    Unlike the Gauss-Legendre version (which is exact here because the
    traction happens to be uniform), a uniform midpoint rule converges at a
    finite algebraic rate and so gives an honest convergence study -- exactly
    what you would see integrating a non-uniform traction from a real code.
    """
    dtheta = np.pi / n_theta
    theta = (np.arange(n_theta) + 0.5) * dtheta          # midpoints in (0, pi)
    _, tz = traction_on_sphere(theta, a, U, mu)
    # dS = 2 pi a^2 sin(theta) dtheta  (phi integrated)
    Fz = np.sum(2 * np.pi * a**2 * np.sin(theta) * tz * dtheta)
    return Fz


def _stress_fd(x, y, z, a, U, mu, eps):
    """Cartesian stress tensor sigma_ij at a point, by finite differences of
    the analytic velocity field.  Used only to validate the *pipeline*
    field -> stress -> traction -> force the way a real solver would."""
    def vel(xi, yi, zi):
        return np.array(velocity_cartesian(xi, yi, zi, a, U))
    X = np.array([x, y, z], float)
    grad = np.zeros((3, 3))                              # grad[i,j] = du_i/dx_j
    for j in range(3):
        dX = np.zeros(3); dX[j] = eps
        up = vel(*(X + dX)); um = vel(*(X - dX))
        grad[:, j] = (up - um) / (2 * eps)
    r, th = np.sqrt(X @ X), None
    th = np.arccos(np.clip(z / r, -1, 1))
    p = pressure(r, th, a, U, mu)
    e = 0.5 * (grad + grad.T)                            # rate of strain
    return -p * np.eye(3) + 2 * mu * e


def drag_by_fd_stress(n_theta=40, r_eval=1.0, a=1.0, U=1.0, mu=1.0, eps=1e-4):
    r"""Drag from the *field*: finite-difference the analytic velocity to get
    the stress on a spherical shell of radius ``r_eval >= a``, dot with the
    outward normal, and integrate.  Converges to 6 pi mu a U as the shell is
    refined.  This mirrors what your project code will do with a *computed*
    (not analytic) velocity field, so it is the most realistic validation.
    """
    mu_c, w = np.polynomial.legendre.leggauss(n_theta)
    theta = np.arccos(mu_c)
    Fz = 0.0
    for th, wi in zip(theta, w):
        n = np.array([np.sin(th), 0.0, np.cos(th)])      # outward normal (x-z plane)
        X = r_eval * n
        sigma = _stress_fd(*X, a, U, mu, eps)
        t = sigma @ n
        Fz += wi * t[2]                                  # z-traction
    return 2 * np.pi * r_eval**2 * Fz


if __name__ == "__main__":
    # quick self-test
    F_num = drag_by_surface_integral(n_theta=64)
    F_exact = stokes_drag_analytic()
    print(f"numerical drag = {F_num:.10f}")
    print(f"analytic drag  = {F_exact:.10f}  (6 pi = {6*np.pi:.10f})")
    print(f"relative error = {abs(F_num-F_exact)/F_exact:.2e}")
```

C.2 stokeslets.py — singularity superposition and pair mobility (Chapters 1–2)

```
"""
stokeslets.py
=============
Singularity building blocks of Stokes flow, and the two-sphere mobility.

This is the second *oracle* module (Chapters 1-2 of the booklet).  It provides:

1.  The elementary singular solutions -- the Stokeslet (Oseen tensor) and the
    potential (source) dipole -- as functions you can superpose.
2.  A reconstruction of the single translating sphere as a Stokeslet + dipole,
    validated against ``sphere_single`` to machine precision.
3.  The Rotne-Prager-Yamakawa (RPY) two-sphere mobility, and the leading
    method-of-reflections result, with a routine that checks one against the
    other.

Everything is exact analysis; there is no linear solve.  The purpose is to give
you closed-form reference values for pair mobilities that your own interaction
code (Chapter 3 and beyond) must reproduce.

Conventions match the booklet: sphere radius ``a``, viscosity ``mu``,
centre-to-centre separation ``d``, unit vector ``d_hat`` from sphere 1 to 2.
"""

from __future__ import annotations
import numpy as np

__all__ = [
    "oseen_tensor", "potential_dipole_tensor", "sphere_velocity_from_singularities",
    "rpy_self", "rpy_cross", "pair_mobility_scalars", "reflection_cross_scalars",
]


# ---------------------------------------------------------------------------
#  Elementary singularities (booklet Eq. 1.10 ff.)
# ---------------------------------------------------------------------------
def oseen_tensor(x, mu=1.0):
    r"""Oseen tensor (Stokeslet) G_ij(x) / (8 pi mu).

    The velocity at ``x`` due to a point force F at the origin is
        u_i = G_ij(x) F_j / (8 pi mu),   G_ij = delta_ij/r + x_i x_j / r^3.
    Returns the 3x3 matrix  M_ij = G_ij / (8 pi mu)  so that  u = M @ F.
    """
    x = np.asarray(x, float)
    r = np.linalg.norm(x)
    I = np.eye(3)
    G = I / r + np.outer(x, x) / r**3
    return G / (8 * np.pi * mu)


def potential_dipole_tensor(x):
```

```
    r"""Potential (source) dipole tensor  D_ij(x) = -delta_ij/r^3 + 3 x_i x_j
        /r^5.

    A force-free singularity (the potential-flow doublet).  The velocity due
        to
    a dipole of vector strength ``d`` is  u_i = D_ij(x) d_j.
    """
    x = np.asarray(x, float)
    r = np.linalg.norm(x)
    I = np.eye(3)
    return -I / r**3 + 3 * np.outer(x, x) / r**5


def sphere_velocity_from_singularities(x, U, a=1.0, mu=1.0):
    r"""Disturbance velocity of a translating sphere, built from
        singularities.

    Booklet Eq. (1.11): a sphere of radius ``a`` translating at velocity ``U
        ``
    (through fluid at rest at infinity) is a Stokeslet of strength
    F = 6 pi mu a U at its centre, MINUS a potential dipole of strength
    (a^3/4) U:

        u_i = [G_ij/(8 pi mu)] (6 pi mu a U_j)  -  (a^3/4) D_ij U_j,

    with D_ij = -delta_ij/r^3 + 3 x_i x_j/r^5.  The minus sign is fixed by
        the
    surface condition u = U on r = a (check it -- the reconstruction test
        does).

    Evaluated at field point ``x`` (sphere centred at origin).  Equals the
    exact field of ``sphere_single`` shifted to the translating frame.
    """
    U = np.asarray(U, float)
    F = 6 * np.pi * mu * a * U                          # Stokeslet strength =
        drag
    u_stokeslet = oseen_tensor(x, mu) @ F
    u_dipole = (a**3 / 4.0) * (potential_dipole_tensor(x) @ U)
    return u_stokeslet - u_dipole


# ------------------------------------------------------------------------
    #
#  Two-sphere mobility: RPY and method of reflections (booklet Sec. 2.3)
        #
# ------------------------------------------------------------------------
    #
def rpy_self(a=1.0, mu=1.0):
    r"""Self-mobility scalar prefactor 1/(6 pi mu a) (x^11 = y^11 = 1)."""
    return 1.0 / (6 * np.pi * mu * a)


def rpy_cross(d, a=1.0, mu=1.0):
    r"""Rotne-Prager-Yamakawa cross-mobility scalars (x^12, y^12), d >= 2a.

    Booklet Eqs. (2.7)-(2.8), in units of the self-mobility 1/(6 pi mu a):

        x^12 = 3a/(2d) -   (a/d)^3
        y^12 = 3a/(4d) + 0.5 (a/d)^3.

    Returns (x12, y12), dimensionless (i.e. divided by 1/(6 pi mu a)).
    """
    d = np.asarray(d, float)
```

```
    s = a / d
    x12 = 1.5 * s - s**3
    y12 = 0.75 * s + 0.5 * s**3
    return x12, y12


def reflection_cross_scalars(d, a=1.0):
    r"""Leading method-of-reflections cross scalars (booklet Eq. 2.6):

        x^12 = 3a/(2d) + O(d^-3),   y^12 = 3a/(4d) + O(d^-3).

    The RPY result should reduce to these as d -> infinity.
    """
    d = np.asarray(d, float)
    s = a / d
    return 1.5 * s, 0.75 * s


def pair_mobility_scalars(d, a=1.0, mu=1.0, model="rpy"):
    r"""Full set of pair-mobility scalars (x11, y11, x12, y12).

    ``model='rpy'`` uses the RPY cross terms; ``model='reflection'`` uses the
    leading 1/d terms.  Self terms are 1 (units of 1/(6 pi mu a)).
    """
    if model == "rpy":
        x12, y12 = rpy_cross(d, a, mu)
    elif model == "reflection":
        x12, y12 = reflection_cross_scalars(d, a)
    else:
        raise ValueError(f"unknown model {model!r}")
    x11 = np.ones_like(np.asarray(d, float))
    y11 = np.ones_like(np.asarray(d, float))
    return x11, y11, x12, y12


# ---------------------------------------------------------------------------
#  Self-tests
# ---------------------------------------------------------------------------
def _reconstruction_error(a=1.0, U=(0.0, 0.0, 1.0), mu=1.3, n=200, seed=0):
    """Max relative error between the singularity reconstruction and the exact
    single-sphere field, over random points outside the sphere."""
    import sys, os
    sys.path.insert(0, os.path.dirname(__file__) or ".")
    import sphere_single as S
    rng = np.random.default_rng(seed)
    U = np.asarray(U, float)
    Umag = np.linalg.norm(U)
    errs = []
    for _ in range(n):
        x = rng.normal(size=3)
        x *= (1.5 + 3 * rng.random()) / np.linalg.norm(x)   # radius in (1.5, 4.5)
        # exact field is for flow past a fixed sphere = uniform U minus the
        # disturbance of a sphere translating at U; here we compare the pure
        # disturbance term directly.
        u_sing = sphere_velocity_from_singularities(x, U, a, mu)
        # A sphere translating at +U through still fluid (u=U on the surface,
        # u->0 at infinity) has velocity field  u_translate = U - u_flowpast,
        # i.e. MINUS the disturbance of uniform flow (stream U) past a fixed
```

```
        # sphere.  (Check the limits: on r=a, u_flowpast=0 -> u_translate=U;
        # at infinity, u_flowpast=U -> u_translate=0.)
        ux, uy, uz = S.velocity_cartesian(*x, a=a, U=Umag)
        u_exact_flow = np.array([ux, uy, uz])
        u_translate = np.array([0.0, 0.0, Umag]) - u_exact_flow
        errs.append(np.linalg.norm(u_sing - u_translate) / Umag)
    return max(errs)


if __name__ == "__main__":
    err = _reconstruction_error()
    print(f"singularity reconstruction max rel. error = {err:.2e}")
    # RPY -> reflection as d grows
    for d in (4, 8, 20, 100):
        x12r, y12r = rpy_cross(d)
        x12f, y12f = reflection_cross_scalars(d)
        print(f"d/a={d:4d}:  x12 RPY={x12r:.5f} refl={x12f:.5f} | "
              f"y12 RPY={y12r:.5f} refl={y12f:.5f}")
```

C.3 `two_sphere_rosa.py` — the two-sphere resistance oracle (Chapter 3)

```
"""
two_sphere_rosa.py
==================
The capstone *oracle*: a two-sphere resistance calculator in the spirit of

    B. Rosa, L.-P. Wang, M.R. Maxey & W.W. Grabowski,
    "An accurate and efficient method for treating aerodynamic interactions
     of cloud droplets", J. Comput. Phys. 230 (2011) 8109-8133.

Rosa et al. compute the hydrodynamic interaction of two settling droplets by
matching an accurate *far-field* multipole (Force-Torque-Stresslet) truncation
to a *near-field* lubrication correction.  This module is a compact, readable
student version of that idea, restricted to the translational problem for two
equal spheres.  It is the trusted reference your project code -- a full
multi-droplet interaction solver, very possibly built with an AI assistant --
should be validated against.

What it computes
----------------
The two normalised translational *resistance functions* for two equal spheres,
as functions of the dimensionless centre-to-centre separation s = d/a (or the
gap xi = s - 2):

    X^A(s)  -- resistance to motion ALONG the line of centres (normal),
    Y^A(s)  -- resistance to motion TRANSVERSE to it (tangential),

each normalised by the single-sphere drag 6 pi mu a.  These are the standard
functions tabulated by Jeffrey & Onishi (J. Fluid Mech. 139 (1984) 261-290).

Strategy (mirrors Rosa et al.)
------------------------------
* FAR field (s large): invert the 2x2 pair mobility built from the RPY tensor
  (Chapter 2) to get resistance.  Accurate to O(s^-6); excellent for gap >~ a.
* NEAR field (xi small): add the lubrication singularities
      X^A ~ 1/(4 xi) + (9/40) ln(1/xi) + ...      (normal, diverges)
```

```
      Y^A ~ (1/6) ln(1/xi) + ...                    (tangential, mild)
  as *corrections* to the far-field value, blended smoothly so neither limit
  is double-counted.

The result is a single callable that is accurate across the whole range of
separations -- exactly the property that makes it useful as an oracle.

Every number here is checked in ``__main__`` against the analytical limits of
Chapters 1-2:  X^A, Y^A -> 1 as s -> infinity (isolated sphere), and the
lubrication divergences as xi -> 0.
"""

from __future__ import annotations
import numpy as np

__all__ = [
    "pair_mobility_matrix", "resistance_from_mobility",
    "lubrication_XA", "lubrication_YA",
    "resistance_XA", "resistance_YA", "resistance_functions",
]


# ---------------------------------------------------------------------------
#  Far field: RPY mobility -> resistance
# ---------------------------------------------------------------------------
def _rpy_scalars(s):
    """RPY self/cross mobility scalars (dimensionless, units 1/(6 pi mu a)) for
    two equal spheres at separation s=d/a.  Self = 1; cross from Chapter 2."""
    inv = 1.0 / s
    x11 = y11 = 1.0
    x12 = 1.5 * inv - inv**3
    y12 = 0.75 * inv + 0.5 * inv**3
    return x11, x12, y11, y12


def pair_mobility_matrix(s, component="normal"):
    r"""2x2 mobility matrix (spheres 1,2) for the chosen component.

    Relates (U1, U2) to (F1, F2) via U = M F, in units where 6 pi mu a = 1:

        [U1]   [ m_self  m_cross ] [F1]
        [U2] = [ m_cross m_self  ] [F2].

    ``component`` is 'normal' (uses x-scalars) or 'transverse' (y-scalars).
    """
    x11, x12, y11, y12 = _rpy_scalars(s)
    if component == "normal":
        m_self, m_cross = x11, x12
    elif component == "transverse":
        m_self, m_cross = y11, y12
    else:
        raise ValueError(component)
    return np.array([[m_self, m_cross], [m_cross, m_self]])


def resistance_from_mobility(s, component="normal"):
    r"""Far-field resistance functions by inverting the pair mobility.

```

```
    For two equal spheres moving together (same velocity, the sedimentation
    mode), the relevant scalar is obtained from the (1,1)+(1,2) combination.
    Here we return the full 2x2 resistance matrix R = M^{-1} (units 6 pi mu a
        ),
    whose diagonal is the self-resistance X^A_11 / Y^A_11.
    """
    M = pair_mobility_matrix(s, component)
    R = np.linalg.inv(M)
    return R


# ------------------------------------------------------------------------
#  Near field: lubrication resistance functions (Jeffrey & Onishi, equal a)
# ------------------------------------------------------------------------
def lubrication_XA(xi):
    r"""Singular part of the normal self-resistance X^A_11 near contact.

    Equal-sphere lubrication expansion (booklet Sec. 2.5):
        X^A_11 ~ 1/(4 xi) + (9/40) ln(1/xi) + O(1),   xi = d/a - 2.
    Returns only the singular (divergent) part; the O(1) constant is absorbed
    by the blend with the far field.
    """
    xi = np.asarray(xi, float)
    return 1.0 / (4.0 * xi) + (9.0 / 40.0) * np.log(1.0 / xi)


def lubrication_YA(xi):
    r"""Singular part of the transverse self-resistance Y^A_11 near contact:
        Y^A_11 ~ (1/6) ln(1/xi) + O(1),   xi = d/a - 2.
    """
    xi = np.asarray(xi, float)
    return (1.0 / 6.0) * np.log(1.0 / xi)


# ------------------------------------------------------------------------
#  Blended oracle: near + far, no double counting
# ------------------------------------------------------------------------
def _blend_weight(xi, xi_switch=0.5, width=0.6):
    """Smooth 0->1 weight that is 1 (pure far field) for large gaps and 0
    (near field dominates) as xi -> 0.  A logistic in log-gap."""
    xi = np.asarray(xi, float)
    return 0.5 * (1.0 + np.tanh((np.log(xi) - np.log(xi_switch)) / width))


def _lub_singular_at(xi, which):
    return lubrication_XA(xi) if which == "X" else lubrication_YA(xi)


def resistance_XA(s):
    r"""Normal self-resistance function X^A_11(s), all separations.

    Blends the far-field (RPY-inverse) value with the near-field lubrication
    singularity so that:
      * s -> infinity  =>  X^A -> 1   (isolated sphere, Chapter 1),
      * xi = s-2 -> 0  =>  X^A ~ 1/(4 xi)  (squeeze-film divergence, Chapter
          2).
    """
```

```
    s = np.asarray(s, float)
    xi = s - 2.0
    R_far = np.array([[np.linalg.inv(pair_mobility_matrix(si, "normal"))[0, 0]]
                      for si in np.atleast_1d(s)]).ravel()
    R_far = R_far.reshape(s.shape) if s.ndim else R_far[0]
    lub = _lub_singular_at(xi, "X")
    w = _blend_weight(xi)
    # near contact (w->0): far value + full lubrication singular part.
    # far field   (w->1): pure far value.
    return R_far + (1.0 - w) * lub


def resistance_YA(s):
    r"""Transverse self-resistance function Y^A_11(s), all separations.

    s -> infinity => 1;  xi -> 0 => (1/6) ln(1/xi)."""
    s = np.asarray(s, float)
    xi = s - 2.0
    R_far = np.array([[np.linalg.inv(pair_mobility_matrix(si, "transverse"))[0, 0]]
                      for si in np.atleast_1d(s)]).ravel()
    R_far = R_far.reshape(s.shape) if s.ndim else R_far[0]
    lub = _lub_singular_at(xi, "Y")
    w = _blend_weight(xi)
    return R_far + (1.0 - w) * lub


def resistance_functions(s):
    """Convenience: return (X^A_11, Y^A_11) at separation(s) s = d/a."""
    return resistance_XA(s), resistance_YA(s)


if __name__ == "__main__":
    import numpy as np
    print("=== limit checks ===")
    for s in (100.0, 20.0, 4.0):
        XA, YA = resistance_functions(np.array(s))
        print(f"s=d/a={s:6.1f}:  X^A={float(XA):.5f}  Y^A={float(YA):.5f}  "
              f"(isolated -> 1)")
    print("--- near contact: X^A should track 1/(4 xi) ---")
    for xi in (0.1, 0.03, 0.01, 0.003):
        s = 2 + xi
        XA = float(resistance_XA(np.array(s)))
        print(f"xi={xi:6.3f}:  X^A={XA:10.3f}   1/(4 xi)={1/(4*xi):10.3f}   "
              f"ratio={XA/(1/(4*xi)):.3f}")
```

C.4 run_all_tests.py — the unified test harness

```
"""
run_all_tests.py
================
One command to validate the whole oracle toolkit.

Run:  python run_all_tests.py

Each check is tied to an analytical result from the reading booklet, and each
prints PASS/FAIL with the measured error.  This is the file to run first when
you clone the code, and the template for how to test your own project code:
every test compares a computation to something you understand.
"""
```

```
13
14 from __future__ import annotations
15 import sys, os
16 import numpy as np
17
18 sys.path.insert(0, os.path.dirname(__file__) or ".")
19 import sphere_single as S
20 import stokeslets as K
21 import two_sphere_rosa as TR
22
23 _results = []
24
25
26 def check(name, ok, detail=""):
27     _results.append(ok)
28     tag = "PASS" if ok else "FAIL"
29     print(f"[{tag}] {name}" + (f"   ({detail})" if detail else ""))
30
31
32 # ------------------------------------------------------------------------
       #
33 #  Chapter 1: single sphere
                                                                  #
34 # ------------------------------------------------------------------------
       #
35 F_exact = 6 * np.pi
36
37 err = abs(S.drag_by_surface_integral(64) - F_exact) / F_exact
38 check("Ch1  Stokes drag by surface integral -> 6 pi mu a U", err < 1e-12,
39       f"rel err {err:.1e}")
40
41 err = abs(S.drag_by_fd_stress(60, r_eval=2.0) - F_exact) / F_exact
42 check("Ch1  drag from FD stress on a shell", err < 1e-7, f"rel err {err:.1e}"
       )
43
44 # midpoint rule converges (coarser -> finer must shrink error)
45 e_coarse = abs(S.drag_by_midpoint_rule(20) - F_exact)
46 e_fine = abs(S.drag_by_midpoint_rule(200) - F_exact)
47 check("Ch1  midpoint quadrature converges with n", e_fine < e_coarse / 50,
48       f"{e_coarse:.1e} -> {e_fine:.1e}")
49
50 # ------------------------------------------------------------------------
       #
51 #  Chapter 1-2: singularity reconstruction
                                            #
52 # ------------------------------------------------------------------------
       #
53 err = K._reconstruction_error(n=300)
54 check("Ch1  sphere = Stokeslet + dipole (surface & field)", err < 1e-12,
55       f"max rel err {err:.1e}")
56
57 # ------------------------------------------------------------------------
       #
58 #  Chapter 2: RPY -> reflection limit, and symmetry
                                  #
59 # ------------------------------------------------------------------------
       #
60 x12r, y12r = K.rpy_cross(200.0)
61 x12f, y12f = K.reflection_cross_scalars(200.0)
62 check("Ch2  RPY -> reflection series as d -> infinity",
63       abs(x12r - x12f) < 1e-4 and abs(y12r - y12f) < 1e-4,
64       f"dx {abs(x12r-x12f):.1e}")
65
```

```
M = TR.pair_mobility_matrix(3.0, "normal")
check("Ch2  pair mobility matrix is symmetric",
      np.allclose(M, M.T, atol=1e-14), "||M-M^T|| < 1e-14")

R = TR.resistance_from_mobility(3.0, "normal")
check("Ch2  resistance matrix (=M^-1) is symmetric",
      np.allclose(R, R.T, atol=1e-12))

# ---------------------------------------------------------------------------
#  Chapter 3: resistance oracle limits                                      #
# ---------------------------------------------------------------------------
XA_far, YA_far = TR.resistance_functions(np.array(100.0))
check("Ch3  X^A, Y^A -> 1 as s -> infinity (isolated sphere)",
      abs(float(XA_far) - 1) < 5e-4 and abs(float(YA_far) - 1) < 5e-4,
      f"X^A={float(XA_far):.4f}")

# near contact: X^A * 4 xi -> 1
xi = 1e-4
XA = float(TR.resistance_XA(np.array(2 + xi)))
ratio = XA * 4 * xi
check("Ch3  X^A ~ 1/(4 xi) near contact (squeeze film)",
      abs(ratio - 1) < 0.05, f"X^A*4xi = {ratio:.3f}")

# both singular terms captured: X^A - [1/(4xi) + (9/40)ln(1/xi)] -> finite const
def jo2(xi): return 0.25/xi + (9/40)*np.log(1/xi)
diffs = [float(TR.resistance_XA(np.array(2+x))) - jo2(x)
         for x in (1e-2, 1e-3, 1e-4)]
spread = max(diffs) - min(diffs)
check("Ch3  X^A captures BOTH singular terms (1/xi and ln)",
      spread < 0.02, f"residual const spread {spread:.1e}")

# ---------------------------------------------------------------------------
print("-" * 60)
n_pass = sum(_results)
n_tot = len(_results)
print(f"{n_pass}/{n_tot} checks passed.")
sys.exit(0 if n_pass == n_tot else 1)
```